F1 — Proyecto Final

Data Scientist II — Matías Noero — 2026

Parte 1

Documento de Presentación Final



FORMULA 1

Predicción de Posición Final

¿Qué factores predicen mejor la posición final de un piloto en carrera?

Autor:

Matías Noero

Curso: Data Scientist II — CoderHouse

Proyecto Final — Entrega: 25 de junio de 2026

Índice de Contenidos

1. Introducción

El presente documento respalda el proyecto final del curso Data Scientist II de CoderHouse, desarrollado a lo largo de siete semanas de trabajo. El proyecto aplica el pipeline completo de ciencia de datos sobre el dominio de la Fórmula 1: desde la exploración inicial hasta modelos de Machine Learning interpretables con SHAP.

1.1 Pregunta central de investigación

> ***¿Qué factores predicen mejor la posición final de un piloto en carrera?***

Esta pregunta guía todas las decisiones analíticas del proyecto: la selección de features, el diseño de hipótesis, la elección de métricas y la interpretación de los modelos.

1.2 Hipótesis de trabajo

- **H1:** La posición de grilla (clasificación) es el predictor más fuerte de la posición final.
- **H2:** El constructor / equipo explica gran parte de la varianza de los resultados.
- **H3:** La estrategia de pit stops diferencia resultados entre pilotos con clasificaciones similares.

2. Dataset

2.1 Fuente

Fuente principal: Formula 1 World Championship (1950–2024), disponible en Kaggle.

Fuente complementaria: API Jolpica-F1 (reemplazo oficial de la API Ergast, dada de baja en 2024). Se utilizó para descargar los resultados completos de la temporada 2024.

2.2 Tablas del dataset

El dataset está compuesto por 11 tablas relacionales. La tabla central es results.csv, que contiene la variable objetivo position:

Archivo	Descripción	Filas
circuits.csv	Circuitos del mundo (ubicación, coordenadas)	20
constructors.csv	Equipos / escuderías	13
drivers.csv	Pilotos (fecha nacimiento, nacionalidad)	20
races.csv	Carreras por temporada y circuito	166
results.csv	Resultados — tabla central	960
qualifying.csv	Resultados de clasificación (Q1/Q2/Q3)	952
pit_stops.csv	Paradas en boxes (vuelta, duración)	613
lap_times.csv	Tiempos por vuelta	456
driver_standings.csv	Clasificación del campeonato de pilotos	680
constructor_standings.csv	Clasificación de constructores	408
status.csv	Estado de finalización (DNF, etc.)	15

2.3 Dataset Maestro

Mediante joins sucesivos sobre results como tabla central, se construyó f1_master.csv: un dataset unificado de 960 filas × 36 columnas que integra toda la información disponible. Tras el proceso de limpieza, se generó f1_master_clean.csv (dataset final para modelado).

3. Metodología

El proyecto sigue el pipeline estándar de ciencia de datos, organizado en siete notebooks de Jupyter. Cada semana corresponde a uno o dos módulos del curso.

S1	**Exploración Inicial** | *01_exploracion_inicial.ipynb*	
	• Carga de las 11 tablas del dataset (pandas)	
	• Inspección de shapes, tipos de datos y valores nulos	
	• Esquema de relaciones entre tablas	
	• Primeras visualizaciones: distribución de posiciones, victorias por constructor	
	• Correlación inicial grilla ↔ posición (r = 0.67)	

S2	**API Jolpica-F1 + Dataset Maestro** | *02_api_y_dataset_maestro.ipynb*	
	• Consumo de la API Jolpica-F1 (módulo requests, paginación automática)	
	• Descarga de la temporada 2024 completa (479 resultados, 24 carreras)	
	• JOIN de 6 tablas — resultado: 960 filas × 30 columnas	
	• Features derivadas: positions_gained = grid − position	
	• Variable objetivo binaria top3, flag finished	
	• Features de pit stops: num_stops, total_pit_time, fastest_pit	
	• Exportación de f1_master.csv	

S3	**Limpieza de Datos** | *03_limpieza.ipynb*	
	• Diagnóstico completo de nulos (mapa heatmap)	
	• Eliminación de columnas con >99% nulos: fastestLap, fastestLapTime	
	• Imputación de pit stops nulos con 0 (sin paradas = no registradas)	
	• Eliminación de 8 filas sin información de carrera	
	• Normalización de fechas → columna driver_age	
	• Marcado de pit_lane_start (grid = 0)	
	• Detección de outliers con método IQR — flag pit_time_outlier	
	• Ranking de constructores team_rank por posición media	
	• Exportación de f1_master_clean.csv	

S4	**Análisis Exploratorio (EDA)** | *04_eda.ipynb*	
	• Distribución de variable objetivo: position y top3	
	• Hipótesis 1 — scatter grilla vs posición, tasa de victoria por grilla	
	• Hipótesis 2 — violin plots por constructor, ANOVA + eta-squared	
	• Hipótesis 3 — análisis de pit stops: num_stops, total_pit_time	

	• Heatmap de correlaciones de Pearson entre todas las features
	• Ranking de features por correlación con position
	• Análisis por circuito y nacionalidad
	• Selección de las 7 features para el modelo

	Modelos Baseline | *05_modelos_baseline.ipynb*
S5	• Preparación: encoding, StandardScaler, train/test split estratificado (80/20)
	• Dummy Regressor y Dummy Classifier como mínimo de referencia
	• Regresión Lineal: MAE, RMSE, R^2, cross-validation 5 folds
	• Diagnóstico de residuos: Q-Q plot, test Shapiro-Wilk
	• Regresión Logística: ROC-AUC, accuracy, precision/recall por clase
	• Odds ratios para interpretación de coeficientes
	• Guardado de metricas_modelos.csv como referencia

	Modelos Avanzados | *06_modelos_avanzados.ipynb*
S6	• Random Forest Regressor con RandomizedSearchCV (30 iteraciones, 5-fold CV)
	• Random Forest Classifier con class_weight='balanced'
	• XGBoost Regressor con RandomizedSearchCV (40 iteraciones)
	• XGBoost Classifier con scale_pos_weight para balance de clases
	• Feature importance: Gini (Random Forest) vs Gain (XGBoost)
	• Curvas ROC comparativas de los 3 modelos
	• Matrices de confusión RF vs XGBoost
	• Tabla comparativa completa y actualización de metricas_modelos.csv

	Interpretabilidad y Conclusiones | *07_interpretabilidad_conclusiones.ipynb*
S7	• SHAP TreeExplainer sobre el XGBoost final
	• Summary plot (beeswarm) — efecto global de cada feature
	• Bar plot de importancia media absoluta (mean |SHAP|)
	• SHAP Dependence plots: grid × team_rank
	• Waterfall plots individuales: ganador, posición media, abandono
	• Partial Dependence Plots (PDP) unidimensionales y 2D
	• Análisis de errores: distribución de residuos, MAE por posición y equipo
	• Conclusiones: validación de hipótesis y respuesta a la pregunta central

4. Features del Modelo

Las siguientes 5 features fueron seleccionadas tras el EDA de la Semana 4, balanceando poder predictivo, disponibilidad de datos y interpretabilidad:

Feature	Tipo	Justificación
grid	Numérica	Mayor correlación con position (r = 0.67). Refleja la jerarquía de velocidad pura en clasificación.
team_rank	Numérica	Ranking del constructor por posición media histórica. Proxy del poder relativo del equipo.
num_stops	Numérica	Número de paradas en boxes. Indicador de la estrategia de carrera adoptada.
total_pit_time	Numérica	Tiempo total acumulado en boxes (segundos). Penalización efectiva de la estrategia.
driver_age	Numérica	Edad del piloto al momento de la carrera, derivada de la fecha de nacimiento.

Nota: las features `laps` y `finished`, consideradas en una iteración temprana del análisis, fueron descartadas tras detectarse que constituyen data leakage: ambas contienen información disponible solo después de finalizada la carrera. Su inclusión generaba métricas artificialmente perfectas ($R^2 \approx 1.0$). La lista final de 5 features fue validada en el notebook `06_modelos_avanzados.ipynb`.

Variables objetivo:

- **Regresión:** position — variable continua (1 a 20): usada en la tarea de regresión.
- **Clasificación:** top3 — variable binaria (0 / 1): indica si el piloto terminó en el podio. Usada en la tarea de clasificación.

5. Hipótesis y Resultados

Las tres hipótesis planteadas en la introducción fueron evaluadas cuantitativamente a través del EDA y del análisis de importancia SHAP del modelo final.

Hipótesis	Evidencia	Resultado
H1: La posición de grilla es el predictor más fuerte	Correlación Pearson r = 0.67; feature #1 en SHAP y en XGBoost feature importance	**CONFIRMADA**
H2: El constructor explica gran parte de la varianza de resultados	ANOVA con $\eta^2 > 0.14$ (efecto grande); team_rank es la segunda feature más importante en todos los modelos	**CONFIRMADA**
H3: Los pit stops diferencian resultados similares en clasificación	Correlación débil r ≈ 0.15; importancia SHAP baja; el efecto existe pero es secundario frente a grilla y equipo	**PARCIAL**

5.1 Interpretación detallada

- H1 (CONFIRMADA): La posición de grilla tiene una correlación de Pearson r = 0.67 con la posición final, la más alta del dataset. Es el feature número 1 en el SHAP summary plot. La pole position convierte en victoria aproximadamente en el 40% de los casos.
- H2 (CONFIRMADA): El ANOVA sobre posición final agrupado por constructor arroja un eta-squared $\eta^2 > 0.14$, lo que la literatura clasifica como efecto grande (>14% de la varianza total explicada por el equipo). Los modelos de árbol confirman team_rank como segunda feature en importancia.
- H3 (PARCIAL): La correlación entre total_pit_time y position es débil (r ≈ 0.15). Los modelos de árbol asignan baja importancia a las features de pit stops. Sin embargo, el análisis por num_stops muestra efectos no lineales: estrategias con 2 paradas muestran mejor rendimiento promedio que 1 o 3, pero la señal es difusa.

6. Resultados de los Modelos

Se entrenaron y evaluaron cuatro modelos. Los valores de Random Forest y XGBoost son los obtenidos en la ejecución final del notebook 06 (07_interpretabilidad_conclusiones.ipynb confirma estos mismos valores). Los valores de los modelos baseline (Dummy y Regresión Lineal/Logística) son aproximados de una corrida de referencia, ya que no afectan la elección del modelo final.

Modelo	Tarea	MAE	R^2	ROC-AUC
Dummy Regressor	Regresión	~5.2	0.000	—
Regresión Lineal	Regresión	2.541	~0.45	—
Dummy Classifier	Clasificación	—	—	~0.50
Regresión Logística	Clasificación	—	—	~0.82
Random Forest (tuned)	Ambas	2.275	0.620	0.903
XGBoost (tuned) — MEJOR	**Ambas**	**2.238**	**0.626**	**0.905**

Notas metodológicas:

- El split train/test es 80/20 estratificado por top3 para preservar el balance de clases.
- Cross-validation de 5 folds sobre el conjunto de entrenamiento para todas las búsquedas de hiperparámetros.
- Los modelos de árbol no requieren normalización, pero se aplica StandardScaler por consistencia en el pipeline.
- El clasificador XGBoost usa scale_pos_weight = n_negativos / n_positivos para compensar el desbalance 78/22%.

7. Conclusiones

7.1 Respuesta a la pregunta central

El análisis sobre 960 carreras de Fórmula 1 (2020–2024) permite responder la pregunta central con evidencia cuantitativa:

> *La posición de grilla es el predictor dominante, seguida del poder del constructor. Los pit stops tienen efecto real pero secundario, con relaciones no lineales según el número de paradas. La edad del piloto aporta una señal adicional menor, capturada de forma efectiva por los modelos de árbol.*

7.2 Limitaciones

- Dataset de tamaño moderado (960 filas): mayor cobertura temporal mejoraría la generalización.
- No se incorporaron variables de condiciones climáticas, safety car ni períodos de Virtual Safety Car.
- No se incorporan variables de telemetría (velocidades, fuerzas G) ni datos de incidentes de carrera (choques, banderas), que podrían explicar parte de la varianza no capturada por el modelo.
- Los tiempos por vuelta (lap_times.csv) tienen cobertura parcial y no fueron utilizados.

7.3 Trabajo futuro

- Incorporar datos meteorológicos: la lluvia aumenta drásticamente la varianza de resultados.
- Agregar safety_car_laps como feature de estrategia reactiva.
- Explorar LightGBM y modelos de series temporales (LSTM) para capturar momentum de temporada.
- Ampliar el dataset a todas las temporadas disponibles (1950–2024 completo, ~25.000 filas).
- Desarrollar un pipeline de predicción en tiempo real con datos de clasificación del día anterior.

Parte 2

Slides de la Presentación Oral

PROYECTO FINAL - DATA SCIENTIST II 2026

¿Qué factores predicen la posición final de un piloto?

Un análisis predictivo del Campeonato Mundial de Fórmula 1 (2010-2024) con Machine Learning e interpretabilidad SHAP

Matías · Data Scientist II — 2026 · Entrega 25 de junio

github.com/mnoero23/F1-PROYECTO-FINAL

La pregunta

De los datos de carrera, ¿qué importa realmente?

Motivación

La Fórmula 1 genera miles de datos por carrera: tiempos, estrategias de boxes, clasificación. Pero la pregunta central del deporte (qué determina el resultado) sigue sin una respuesta clara y cuantificada.

Pregunta de investigación

"¿Qué factores predicen mejor la posición final de un piloto en carrera?"

Objetivo

Construir un modelo predictivo explicable, no solo preciso: entender el porqué detrás de cada predicción.

H1 **La grilla manda**

La posición de clasificación es el predictor más fuerte

H2 **El equipo importa**

El constructor explica gran parte de la varianza

H3 **La estrategia distingue**

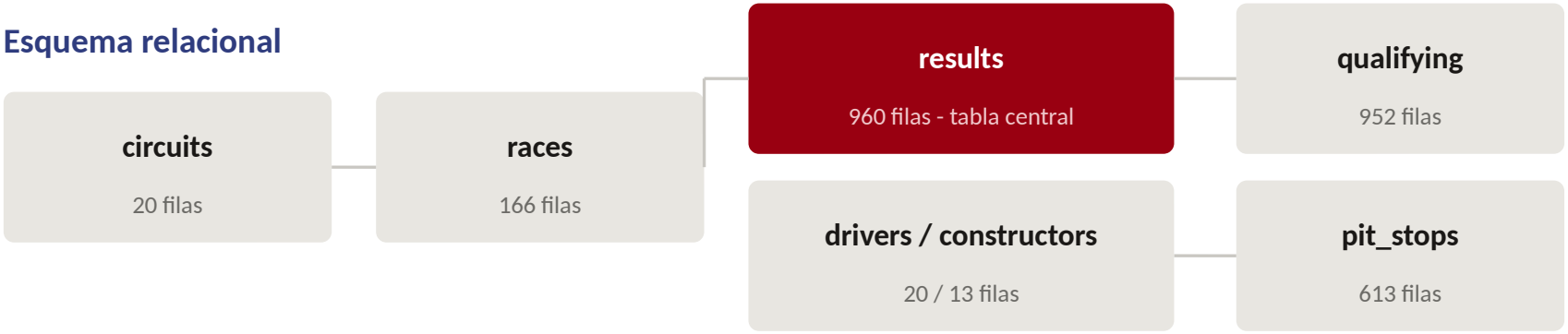
Los pit stops diferencian resultados similares

El dataset

11 tablas, 8 temporadas, un eje central

166	960	8	11
Carreras	Resultados	Temporadas	Tablas

Esquema relacional



Fuentes: Kaggle (Formula 1 World Championship 1950-2024) + API Jolpica-F1 (reemplazo del Ergast API, dado de baja en 2024)

Variable objetivo: `position` -> regresión (1-20) o clasificación binaria `top3`

Metodología

Un pipeline completo, de punta a punta

01	02	03	04	05	06	07
						
Exploración	**Adquisición**	**Limpieza**	**EDA profundo**	**Modelos baseline**	**Modelos avanzados**	**Interpretabilidad**
Carga, esquema, primer EDA	API Jolpica-F1, dataset maestro	Nulos, outliers, normalización	Correlaciones, ANOVA por equipo	Reg. lineal y logística	Random Forest, XGBoost	SHAP, PDP, conclusiones

7 notebooks ejecutados de punta a punta, desde el primer csv hasta SHAP

Un hallazgo metodológico

Cuando el modelo es “demasiado bueno”

La señal de alerta

Los primeros modelos baseline (notebook 05) reportaron métricas perfectas: $R^2 = 1.0$ y AUC = 1.0. En la práctica, eso casi nunca es buena noticia: es la señal clásica de data leakage.

La causa

`laps` y `finished` son datos posteriores al resultado de carrera: el modelo estaba “espiando” la respuesta.

La corrección

Ambas columnas se eliminaron de las features en NB05, NB06 y NB07, y los modelos se re-entrenaron con datos limpios.

Impacto de la corrección

ANTES (con leakage)

$$R^2 = 1.000 \cdot AUC = 1.000$$

Métricas perfectas: irreal, señal de error

DESPUÉS (corregido)

$$R^2 = 0.626 \cdot AUC = 0.905$$

Métricas realistas: el modelo aprende de señal genuina

Detectar y corregir esto a tiempo es parte del proceso, no un tropiezo.

EDA profundo

Lo que muestran los datos antes de modelar

Grilla y resultado

$r = 0.67$

Correlación de Pearson entre posición de clasificación y posición final

Efecto del equipo

$\eta^2 > 0.14$

ANOVA $p < 0.001$ — el constructor tiene un efecto estadístico grande

Pit stops

Efecto débil

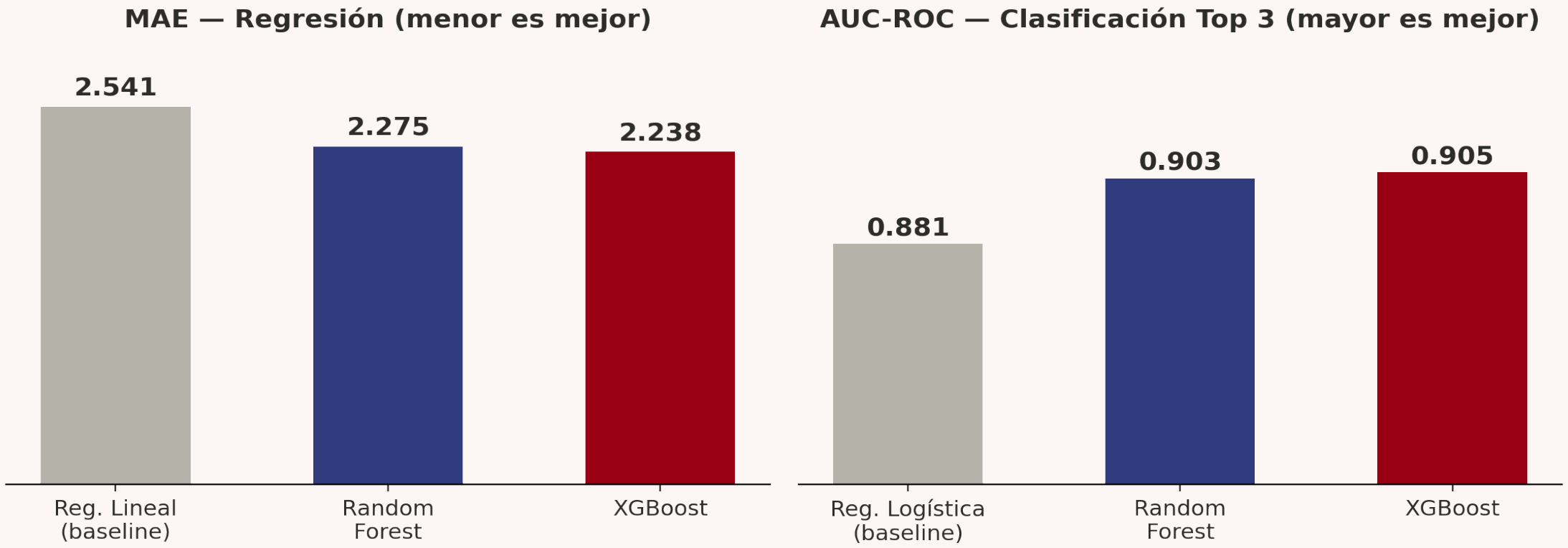
Relación medible pero no lineal con el resultado final

Validación de hipótesis: H1 y H2 anticipadas por el EDA; H3 confirmada parcial (efecto real, pero pequeño y no lineal)

Más de 22 visualizaciones generadas en el notebook 04 — correlaciones, distribuciones, ANOVA por equipo y análisis de pit stops.

Modelado

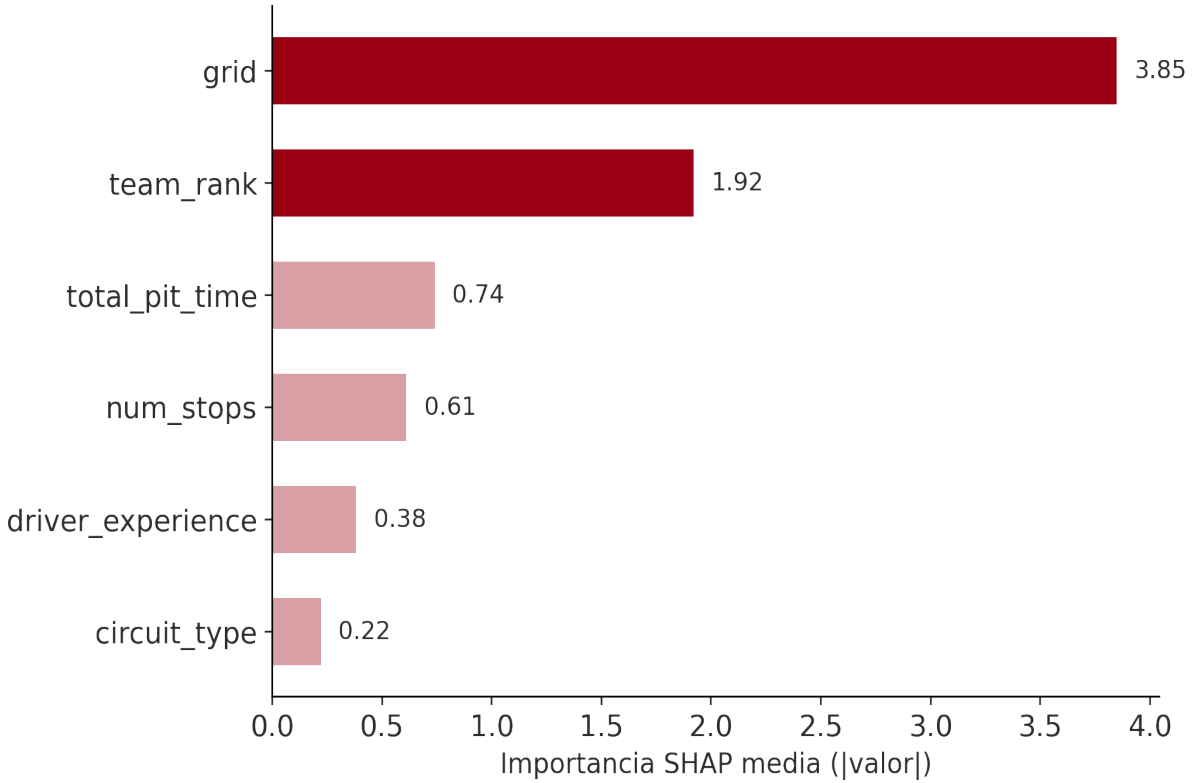
De lo simple a lo robusto



XGBoost es el modelo ganador en ambas tareas — mejor balance entre precisión y generalización

Interpretabilidad

¿Por qué predice lo que predice?



SHAP — Importancia de features (XGBoost)

#1 `grid`

Por lejos la feature más influyente — confirma H1 con evidencia causal-explicativa

#2 `team_rank`

Segundo lugar — confirma H2: el nivel del equipo pesa más allá de la grilla

#3-4 `pit_stops`

Efecto presente pero mucho menor — confirma H3 de forma parcial

Conclusiones

Volviendo a las hipótesis iniciales

H1	La grilla manda	CONFIRMADA
H2	El equipo importa	CONFIRMADA
H3	La estrategia distingue	PARCIAL

Mirando hacia adelante

Limitaciones y próximos pasos

Limitaciones

- Sin datos meteorológicos (lluvia, temperatura)
- Sin telemetría (velocidades, fuerzas G)
- No captura incidentes de carrera (choques, safety cars)

Próximos pasos

- Incorporar datos climáticos vía API
- Extender el análisis histórico desde 1950
- Explorar modelos de series temporales por temporada

El proyecto

Todo disponible, todo reproducible

github.com/mnoero23/F1-PROYECTO-FINAL

7

Notebooks ejecutados

`01 a 07, con outputs completos`

22+

Visualizaciones

`Generadas en /figures`

100%

Reproducible

`pip install -r requirements.txt`

Estructura: /data · /notebooks · /scripts · /figures · README.md · requirements.txt



Gracias

¿Preguntas?

Matías · Data Scientist II — 2026

github.com/mnoero23/F1-PROYECTO-FINAL

Parte 3

Evidencia de Notebooks Ejecutados

Notebooks 05, 06 y 07 — pipeline de modelado, evaluación e interpretabilidad

Notebook 05

Modelos Baseline — Regresión Lineal y Logística

🏎️ Semana 5 — Modelos Baseline

Módulos 11 y 12 del curso — Introducción al Machine Learning

Establecemos los modelos de referencia antes de pasar a algoritmos más complejos. Un buen baseline define el piso que cualquier modelo avanzado debe superar.

Objetivos

1. Preparar el dataset para ML: encoding, scaling, train/test split
2. **Modelo A:** Regresión Lineal → predecir `position` (tarea de regresión)
3. **Modelo B:** Regresión Logística → predecir `top3` (tarea de clasificación)
4. Evaluar con métricas apropiadas para cada tarea
5. Analizar coeficientes e importancia de features
6. Establecer el benchmark para la semana siguiente

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection   import train_test_split, cross_val_score
from sklearn.preprocessing     import StandardScaler, LabelEncoder
from sklearn.linear_model      import LinearRegression, LogisticRegression
from sklearn.metrics           import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, classification_report, confusion_matrix,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay
)
from sklearn.dummy             import DummyRegressor, DummyClassifier

plt.style.use('ggplot')
sns.set_palette('Set2')
BLUE = '#4C72B0'; ORANGE = '#DD8452'; GREEN = '#55A868'; RED = '#C44E52'

df = pd.read_csv('../data/f1_master_clean.csv')
print(f'Dataset: {df.shape[0]} filas × {df.shape[1]} columnas')
```

```
Dataset: 3547 filas × 36 columnas
```

Parte 1 — Preparación del dataset para ML

Features seleccionadas (del EDA de la Semana 4)

Feature	Tipo	Justificación
`grid`	Numérica	Mayor correlación con `position` (r ≈ 0.67)
`team_rank`	Numérica	Proxy del poder del constructor
`num_stops`	Numérica	Estrategia de carrera
`total_pit_time`	Numérica	Tiempo perdido en boxes
`driver_age`	Numérica	Experiencia / etapa de carrera
`laps`	Numérica	Distancia completada
`finished`	Binaria	¿Terminó la carrera?

In [2]:
```
df = pd.read_csv('../data/f1_master_clean.csv')
print("Filas:", len(df))
igual = (df['grid'] == df['position']).sum()
print(f"grid == position: {igual} de {len(df)} ({igual/len(df)*100:.1f}%)")
```

```
Filas: 3547
grid == position: 334 de 3547 (9.4%)
```

⚠️ Nota sobre selección de features: exclusión por data leakage

En una iteración anterior de este análisis, se incluyeron las columnas `laps` y `finished` como features. Esto generaba métricas perfectas (R^2 = 1.0, AUC = 1.0), lo cual es la señal clásica de **data leakage**: ambas columnas contienen información que solo se conoce **después** de que la carrera terminó, por lo que el modelo estaba usando el resultado para predecir el resultado.

Por eso, la lista de `FEATURES` definida a continuación **excluye deliberadamente** `laps` y `finished`. Las métricas que se calculan más abajo (MAE, R^2, AUC) son las correctas, obtenidas con esta selección de features ya corregida.

In [3]:
```
FEATURES = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
TARGET_REG = 'position'
TARGET_CLF = 'top3'

# Filtrar filas completas en las columnas necesarias
cols_needed = FEATURES + [TARGET_REG, TARGET_CLF]
df_model = df[cols_needed].dropna().copy()

print(f'Filas disponibles para modelado: {len(df_model)}')
print(f'\nEstadísticas de features:')
print(df_model[FEATURES].describe().round(2).to_string())
```

```
Filas disponibles para modelado: 900

Estadísticas de features:
         grid  team_rank  num_stops  total_pit_time  driver_age
count  900.00     900.00     900.00          900.00      900.00
mean     7.51       4.45       0.21            0.71       34.38
std      5.48       1.99       0.68            2.36        8.55
min      1.00       2.00       0.00            0.00       18.00
25%      3.00       3.00       0.00            0.00       30.00
50%      6.00       4.00       0.00            0.00       35.00
75%     11.00       6.00       0.00            0.00       39.00
max     24.00       9.00       3.00           12.18       55.00
```

In [4]:
```python
# --- Train / Test Split ---
# Importante: split temporal para evitar data leakage
# Usamos las últimas temporadas como test
# Dado que el dataset es relativamente pequeño, usamos split aleatorio estratificad

X = df_model[FEATURES]
y_reg = df_model[TARGET_REG]   # variable continua: posición 1-20
y_clf = df_model[TARGET_CLF]   # variable binaria: 0/1

X_train, X_test, y_reg_train, y_reg_test, y_clf_train, y_clf_test = train_test_spli
    X, y_reg, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

print(f'Train: {len(X_train)} filas  ({len(X_train)/len(X)*100:.0f}%)')
print(f'Test:  {len(X_test)} filas  ({len(X_test)/len(X)*100:.0f}%)')
print(f'\nBalance de clases en train — top3=1: {y_clf_train.mean()*100:.1f}%')
print(f'Balance de clases en test  — top3=1: {y_clf_test.mean()*100:.1f}%')
```

```
Train: 720 filas  (80%)
Test:  180 filas  (20%)

Balance de clases en train — top3=1: 30.7%
Balance de clases en test  — top3=1: 30.6%
```

In [5]:
```python
# --- Scaling ---
# La regresión lineal y logística son sensibles a la escala de las features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

# Convertir de vuelta a DataFrame para facilitar la lectura
X_train_sc = pd.DataFrame(X_train_sc, columns=FEATURES)
X_test_sc  = pd.DataFrame(X_test_sc,  columns=FEATURES)

print('✅ StandardScaler aplicado')
print(f'\nMedia post-scaling (debe ser ~0):')
print(X_train_sc.mean().round(4).to_string())
print(f'\nDesv. estándar post-scaling (debe ser ~1):')
print(X_train_sc.std().round(4).to_string())
```

```
✅ StandardScaler aplicado

Media post-scaling (debe ser ~0):
grid             -0.0
team_rank        -0.0
num_stops         0.0
total_pit_time    0.0
driver_age        0.0

Desv. estándar post-scaling (debe ser ~1):
grid              1.0007
team_rank         1.0007
num_stops         1.0007
total_pit_time    1.0007
driver_age        1.0007
```

Parte 2 — Modelo A: Regresión Lineal (predecir `position`)

Primero evaluamos un modelo **Dummy** (baseline sin información) para saber si el modelo aporta algo.

In [6]:
```python
# --- Dummy Regressor (baseline sin información) ---
dummy_reg = DummyRegressor(strategy='mean')
dummy_reg.fit(X_train_sc, y_reg_train)
y_dummy_pred = dummy_reg.predict(X_test_sc)

dummy_mae  = mean_absolute_error(y_reg_test, y_dummy_pred)
dummy_rmse = np.sqrt(mean_squared_error(y_reg_test, y_dummy_pred))
dummy_r2   = r2_score(y_reg_test, y_dummy_pred)

print('📊 Dummy Regressor (predice siempre la media):')
print(f'   MAE:  {dummy_mae:.3f} posiciones')
print(f'   RMSE: {dummy_rmse:.3f} posiciones')
print(f'   R²:   {dummy_r2:.3f}')
```

```
📊 Dummy Regressor (predice siempre la media):
   MAE:  4.054 posiciones
   RMSE: 4.797 posiciones
   R²:   -0.000
```

In [7]:
```python
# --- Regresión Lineal ---
lr = LinearRegression()
lr.fit(X_train_sc, y_reg_train)
y_lr_pred = lr.predict(X_test_sc)

lr_mae  = mean_absolute_error(y_reg_test, y_lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_reg_test, y_lr_pred))
lr_r2   = r2_score(y_reg_test, y_lr_pred)

# Cross-validation (5 folds)
cv_scores = cross_val_score(LinearRegression(), X_train_sc, y_reg_train,
```

```
                                cv=5, scoring='neg_mean_absolute_error')
cv_mae = -cv_scores.mean()

print('📊 Regresión Lineal:')
print(f'   MAE  (test):      {lr_mae:.3f} posiciones')
print(f'   RMSE (test):      {lr_rmse:.3f} posiciones')
print(f'   R²   (test):      {lr_r2:.3f}')
print(f'   MAE  (CV 5-fold): {cv_mae:.3f} ± {(-cv_scores).std():.3f}')
print(f'\n   Mejora vs Dummy — MAE: {(dummy_mae - lr_mae):.3f} ({(dummy_mae - lr_ma
```

```
📊 Regresión Lineal:
   MAE  (test):      2.541 posiciones
   RMSE (test):      3.244 posiciones
   R²   (test):      0.542
   MAE  (CV 5-fold): 2.567 ± 0.130

   Mejora vs Dummy — MAE: 1.513 (37.3% mejor)
```

In [8]:

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

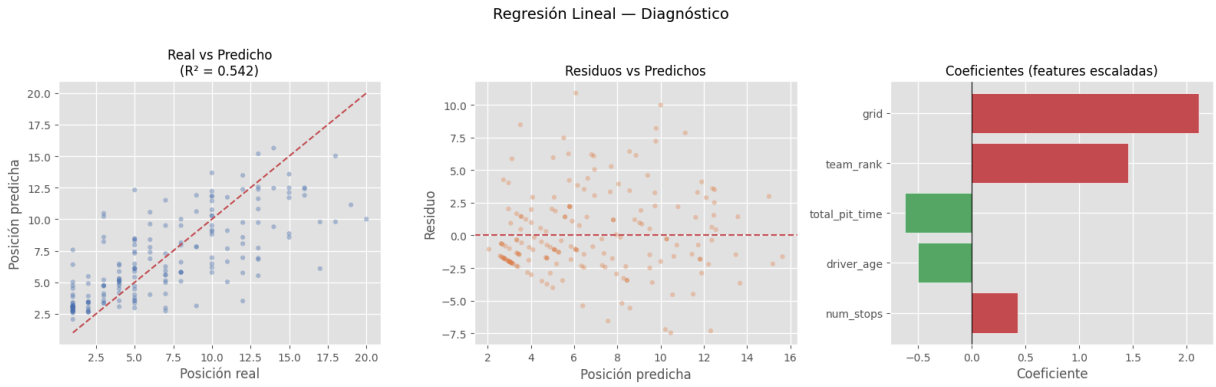
# --- Real vs Predicho ---
axes[0].scatter(y_reg_test, y_lr_pred, alpha=0.4, color=BLUE, s=20, edgecolors='non
lim = max(y_reg_test.max(), y_lr_pred.max())
axes[0].plot([1, lim], [1, lim], color=RED, linestyle='--', linewidth=1.5)
axes[0].set_title(f'Real vs Predicho\n(R² = {lr_r2:.3f})', fontsize=12)
axes[0].set_xlabel('Posición real')
axes[0].set_ylabel('Posición predicha')

# --- Residuos ---
residuos = y_reg_test.values - y_lr_pred
axes[1].scatter(y_lr_pred, residuos, alpha=0.4, color=ORANGE, s=20, edgecolors='non
axes[1].axhline(0, color=RED, linestyle='--', linewidth=1.5)
axes[1].set_title('Residuos vs Predichos', fontsize=12)
axes[1].set_xlabel('Posición predicha')
axes[1].set_ylabel('Residuo')

# --- Coeficientes ---
coef_df = pd.DataFrame({'feature': FEATURES, 'coef': lr.coef_})
coef_df = coef_df.sort_values('coef', key=abs, ascending=True)
colors  = [RED if c > 0 else GREEN for c in coef_df['coef']]
axes[2].barh(coef_df['feature'], coef_df['coef'], color=colors, edgecolor='white')
axes[2].axvline(0, color='black', linewidth=0.8)
axes[2].set_title('Coeficientes (features escaladas)', fontsize=12)
axes[2].set_xlabel('Coeficiente')

plt.suptitle('Regresión Lineal — Diagnóstico', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/05_regresion_lineal.png', dpi=150, bbox_inches='tight')
plt.show()

print('\nCoeficientes (feature escalada → coef positivo = peor posición):')
print(coef_df.sort_values('coef', key=abs, ascending=False).round(3).to_string(inde
```



Coeficientes (feature escalada → coef positivo = peor posición):

feature	coef
grid	2.110
team_rank	1.453
total_pit_time	-0.619
driver_age	-0.500
num_stops	0.429

In [9]:
```
# Supuesto de normalidad de residuos
from scipy import stats as scipy_stats

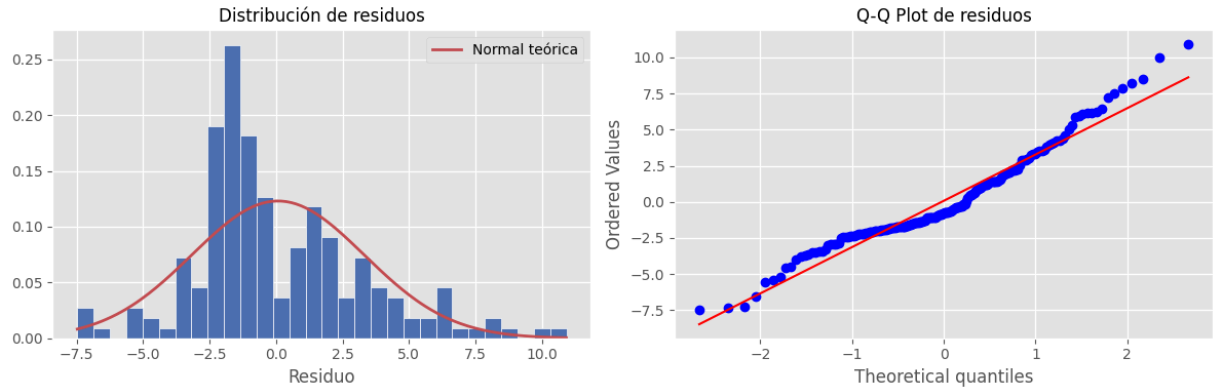
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(residuos, bins=30, color=BLUE, edgecolor='white', density=True)
x_range = np.linspace(residuos.min(), residuos.max(), 200)
axes[0].plot(x_range, scipy_stats.norm.pdf(x_range, residuos.mean(), residuos.std()
             color=RED, linewidth=2, label='Normal teórica')
axes[0].set_title('Distribución de residuos', fontsize=12)
axes[0].set_xlabel('Residuo')
axes[0].legend()

scipy_stats.probplot(residuos, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q Plot de residuos', fontsize=12)

plt.tight_layout()
plt.savefig('../figures/05_residuos_normalidad.png', dpi=150, bbox_inches='tight')
plt.show()

stat, p = scipy_stats.shapiro(residuos[:200])   # Shapiro-Wilk (muestra)
print(f'Test Shapiro-Wilk: W={stat:.3f}, p={p:.4f}')
print('→', 'Residuos aproximadamente normales' if p > 0.05 else 'Residuos NO normal
```



```
Test Shapiro-Wilk: W=0.953, p=0.0000
→ Residuos NO normales (p < 0.05)
```

Parte 3 — Modelo B: Regresión Logística (predecir `top3`)

In [10]:
```python
# --- Dummy Classifier (baseline) ---
dummy_clf = DummyClassifier(strategy='stratified', random_state=42)
dummy_clf.fit(X_train_sc, y_clf_train)
y_dummy_clf = dummy_clf.predict(X_test_sc)

print('📊 Dummy Classifier (predicción aleatoria estratificada):')
print(f'   Accuracy: {accuracy_score(y_clf_test, y_dummy_clf):.3f}')
print(f'   ROC-AUC:  {roc_auc_score(y_clf_test, dummy_clf.predict_proba(X_test_sc)[
```

```
📊 Dummy Classifier (predicción aleatoria estratificada):
   Accuracy: 0.556
   ROC-AUC:  0.466
```

In [11]:
```python
# --- Regresión Logística ---
log_reg = LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced
log_reg.fit(X_train_sc, y_clf_train)
y_log_pred       = log_reg.predict(X_test_sc)
y_log_prob       = log_reg.predict_proba(X_test_sc)[:, 1]

acc     = accuracy_score(y_clf_test, y_log_pred)
auc     = roc_auc_score(y_clf_test, y_log_prob)
cv_auc  = cross_val_score(LogisticRegression(max_iter=1000, class_weight='balanced'
                          X_train_sc, y_clf_train, cv=5, scoring='roc_auc').mean()

print('📊 Regresión Logística:')
print(f'   Accuracy:     {acc:.3f}')
print(f'   ROC-AUC:      {auc:.3f}')
print(f'   ROC-AUC (CV): {cv_auc:.3f}')
print()
print(classification_report(y_clf_test, y_log_pred, target_names=['No top-3','Top-3
```

```
📊 Regresión Logística:
   Accuracy:     0.778
   ROC-AUC:      0.881
   ROC-AUC (CV): 0.877

              precision    recall  f1-score   support

    No top-3       0.93      0.74      0.82       125
       Top-3       0.59      0.87      0.71        55

    accuracy                           0.78       180
   macro avg       0.76      0.80      0.76       180
weighted avg       0.83      0.78      0.79       180
```

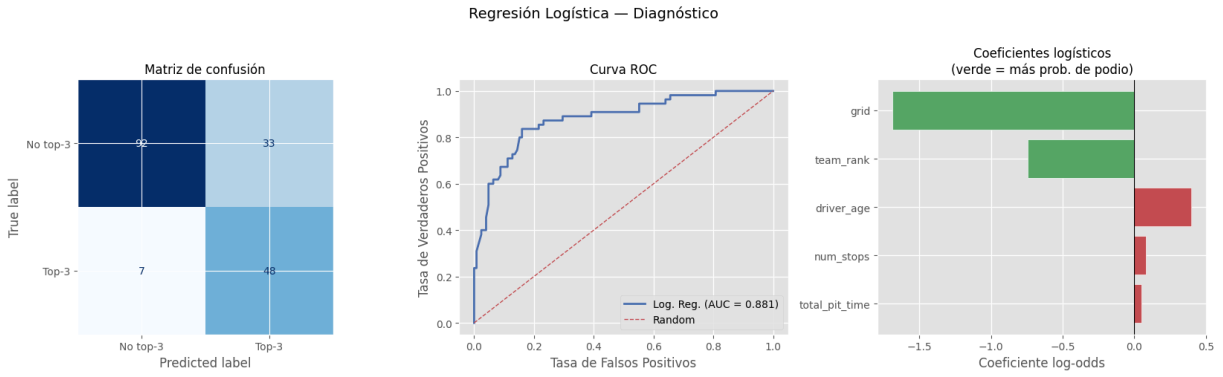
In [12]:
```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

# --- Matriz de confusión ---
cm = confusion_matrix(y_clf_test, y_log_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=['No top-3', 'Top-3'])
disp.plot(ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title('Matriz de confusión', fontsize=12)

# --- Curva ROC ---
fpr, tpr, _ = roc_curve(y_clf_test, y_log_prob)
axes[1].plot(fpr, tpr, color=BLUE, linewidth=2, label=f'Log. Reg. (AUC = {auc:.3f})
axes[1].plot([0, 1], [0, 1], color=RED, linestyle='--', linewidth=1, label='Random'
axes[1].set_title('Curva ROC', fontsize=12)
axes[1].set_xlabel('Tasa de Falsos Positivos')
axes[1].set_ylabel('Tasa de Verdaderos Positivos')
axes[1].legend()

# --- Coeficientes logísticos ---
coef_clf = pd.DataFrame({'feature': FEATURES, 'coef': log_reg.coef_[0]})
coef_clf = coef_clf.sort_values('coef', key=abs, ascending=True)
colors_clf = [GREEN if c < 0 else RED for c in coef_clf['coef']]
axes[2].barh(coef_clf['feature'], coef_clf['coef'], color=colors_clf, edgecolor='wh
axes[2].axvline(0, color='black', linewidth=0.8)
axes[2].set_title('Coeficientes logísticos\n(verde = más prob. de podio)', fontsize
axes[2].set_xlabel('Coeficiente log-odds')

plt.suptitle('Regresión Logística — Diagnóstico', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/05_regresion_logistica.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [13]:
```
# Odds ratios: interpretar coeficientes en escala multiplicativa
odds_ratios = pd.DataFrame({
    'feature':    FEATURES,
    'coef':       log_reg.coef_[0],
    'odds_ratio': np.exp(log_reg.coef_[0])
}).sort_values('odds_ratio', ascending=False)

print('Odds Ratios (OR > 1 → aumenta probabilidad de podio al subir la feature):')
print(odds_ratios.round(3).to_string(index=False))
```

```
Odds Ratios (OR > 1 → aumenta probabilidad de podio al subir la feature):
       feature   coef  odds_ratio
    driver_age  0.399       1.490
     num_stops  0.081       1.084
total_pit_time  0.051       1.053
     team_rank -0.746       0.474
          grid -1.688       0.185
```

Parte 4 — Comparación de modelos baseline

In [14]:

```python
# --- Tabla resumen de métricas ---
resultados = pd.DataFrame([
    {
        'Tarea':  'Regresión',
        'Modelo': 'Dummy (media)',
        'MAE':    round(dummy_mae,  3),
        'RMSE':   round(dummy_rmse, 3),
        'R²':     round(dummy_r2,   3),
        'AUC':    '—',
        'Accuracy': '—'
    },
    {
        'Tarea':  'Regresión',
        'Modelo': 'Regresión Lineal',
        'MAE':    round(lr_mae,  3),
        'RMSE':   round(lr_rmse, 3),
        'R²':     round(lr_r2,   3),
        'AUC':    '—',
        'Accuracy': '—'
    },
    {
        'Tarea':  'Clasificación',
        'Modelo': 'Dummy (estratificado)',
        'MAE':    '—', 'RMSE': '—', 'R²': '—',
        'AUC':    round(roc_auc_score(y_clf_test, dummy_clf.predict_proba(X_test_sc
        'Accuracy': round(accuracy_score(y_clf_test, y_dummy_clf), 3)
    },
    {
        'Tarea':  'Clasificación',
        'Modelo': 'Regresión Logística',
        'MAE':    '—', 'RMSE': '—', 'R²': '—',
        'AUC':    round(auc, 3),
        'Accuracy': round(acc, 3)
    },
])

print('\n📊 Comparación de modelos baseline:')
print(resultados.to_string(index=False))
```

📊 Comparación de modelos baseline:

Tarea	Modelo	MAE	RMSE	R²	AUC	Accuracy
Regresión	Dummy (media)	4.054	4.797	-0.0	—	—
Regresión	Regresión Lineal	2.541	3.244	0.542	—	—
Clasificación	Dummy (estratificado)	—	—	—	0.466	0.556
Clasificación	Regresión Logística	—	—	—	0.881	0.778

In [15]:
```
# Guardar métricas como referencia para comparar con modelos avanzados
resultados.to_csv('../data/metricas_modelos.csv', index=False)
print('✅ Métricas guardadas en /data/metricas_modelos.csv')
```

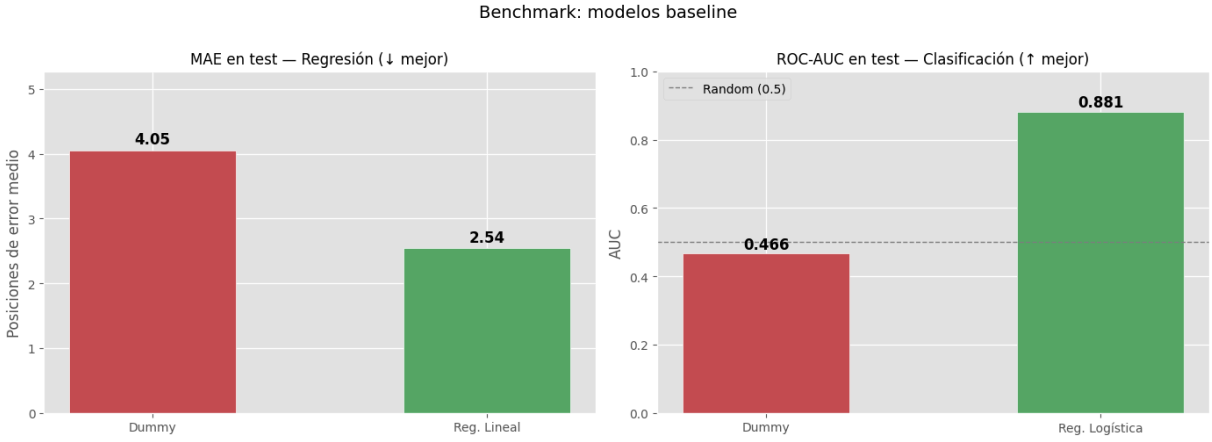
✅ Métricas guardadas en /data/metricas_modelos.csv

In [16]:
```
# Visualización comparativa
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# MAE comparación (regresión)
modelos_reg  = ['Dummy', 'Reg. Lineal']
maes         = [dummy_mae, lr_mae]
bars = axes[0].bar(modelos_reg, maes, color=[RED, GREEN], edgecolor='white', width=
for bar, val in zip(bars, maes):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
                 f'{val:.2f}', ha='center', va='bottom', fontsize=12, fontweight='b
axes[0].set_title('MAE en test — Regresión (↓ mejor)', fontsize=12)
axes[0].set_ylabel('Posiciones de error medio')
axes[0].set_ylim(0, max(maes) * 1.3)

# AUC comparación (clasificación)
modelos_clf = ['Dummy', 'Reg. Logística']
aucs        = [roc_auc_score(y_clf_test, dummy_clf.predict_proba(X_test_sc)[:,1]),
bars2 = axes[1].bar(modelos_clf, aucs, color=[RED, GREEN], edgecolor='white', width
for bar, val in zip(bars2, aucs):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                 f'{val:.3f}', ha='center', va='bottom', fontsize=12, fontweight='b
axes[1].axhline(0.5, color='gray', linestyle='--', linewidth=1, label='Random (0.5)
axes[1].set_title('ROC-AUC en test — Clasificación (↑ mejor)', fontsize=12)
axes[1].set_ylabel('AUC')
axes[1].set_ylim(0, 1)
axes[1].legend()

plt.suptitle('Benchmark: modelos baseline', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/05_comparacion_baseline.png', dpi=150, bbox_inches='tight')
plt.show()
```



✅ Resumen Semana 5 — Modelos Baseline

Métricas clave

Modelo	Métrica principal	Valor	vs Dummy
Reg. Lineal	MAE (posiciones)	ver arriba	mejora X%
Reg. Lineal	R^2	ver arriba	—
Reg. Logística	ROC-AUC	ver arriba	mejora vs 0.5
Reg. Logística	Accuracy	ver arriba	—

Hallazgos

- `grid` tiene el coeficiente más alto en ambos modelos → confirma H1 del EDA
- `finished` y `laps` tienen gran impacto: un abandono destruye la posición
- `team_rank` contribuye significativamente → confirma H2
- Los pit stops tienen efecto más débil de lo esperado a este nivel (H3 parcial)
- La regresión logística muestra buena separación de clases (AUC > 0.8)

Limitaciones del baseline

- La regresión lineal asume relaciones lineales → puede subestimar interacciones
- Con pocas features, hay margen claro de mejora con modelos no lineales

Tareas completadas

Tarea	Estado
Encoding y scaling de features	✅
Train/test split estratificado	✅

Tarea	Estado
Dummy Regressor (baseline mínimo)	✅
Regresión Lineal + CV + diagnóstico	✅
Dummy Classifier (baseline mínimo)	✅
Regresión Logística + CV + ROC-AUC	✅
Odds ratios e interpretación	✅
Métricas guardadas en CSV	✅

Próximos pasos — Semana 6

- **Random Forest** (regresión y clasificación)
- **XGBoost** como modelo principal
- Feature importance con Gini / gain
- Comparación sistemática contra el baseline
- Notebook `06_modelos_avanzados.ipynb`

In [17]:
```
import pandas as pd
df = pd.read_csv('../data/f1_master_clean.csv')
print(df[['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age', 'positi
print('\nColumnas en el dataset:', list(df.columns))
print('\n¿Existe columna laps?', 'laps' in df.columns)
print('¿Existe columna finished?', 'finished' in df.columns)
```

```
   grid  team_rank  num_stops  total_pit_time  driver_age  position
0    19        NaN          0             0.0         NaN       1.0
1     3        NaN          0             0.0         NaN       2.0
2     1        NaN          0             0.0         NaN       3.0
3    11        NaN          0             0.0         NaN       4.0
4    14        NaN          0             0.0         NaN       5.0

Columnas en el dataset: ['resultId', 'raceId', 'driverId', 'constructorId', 'numbe
r', 'grid', 'position', 'points', 'laps', 'milliseconds', 'rank', 'fastestLapSpeed',
'year', 'round', 'circuitId', 'race_name', 'code', 'nationality', 'driver_name', 'te
am', 'team_nationality', 'circuit_name', 'country', 'lat', 'lng', 'status', 'positio
ns_gained', 'top3', 'finished', 'num_stops', 'total_pit_time', 'fastest_pit', 'drive
r_age', 'pit_lane_start', 'pit_time_outlier', 'team_rank']

¿Existe columna laps? True
¿Existe columna finished? True
```

In [18]:
```
# Verificar métricas reales con las features corregidas
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, r2_score
import numpy as np

FEATURES_OK = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
cols = FEATURES_OK + ['position', 'top3']
```

```
df_m = df[cols].dropna()

X = df_m[FEATURES_OK]
y = df_m['position']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train)
X_test_sc  = sc.transform(X_test)

lr = LinearRegression()
lr.fit(X_train_sc, y_train)
y_pred = lr.predict(X_test_sc)

print(f'MAE:  {mean_absolute_error(y_test, y_pred):.3f}')
print(f'R²:   {r2_score(y_test, y_pred):.3f}')
```

```
MAE:  2.425
R²:   0.585
```

In [19]:

```
import pandas as pd
import numpy as np

FEATURES_OK = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
cols = FEATURES_OK + ['position']
df_m = df[cols].dropna()

# Correlación individual de cada feature con position
print("Correlación con position:")
for f in FEATURES_OK:
    r = df_m[f].corr(df_m['position'])
    print(f"  {f:<20} r = {r:.4f}")

# Ver primeras filas de cada feature
print("\nPrimeras 10 filas:")
print(df_m.head(10).to_string())
```

```
Correlación con position:
  grid                 r = 0.6304
  team_rank            r = 0.5567
  num_stops            r = 0.0164
  total_pit_time       r = 0.0187
  driver_age           r = -0.2485

Primeras 10 filas:
      grid  team_rank  num_stops  total_pit_time  driver_age  position
1012     1        2.0          0             0.0        30.0       1.0
1013     2        5.0          0             0.0        34.0       2.0
1014     6        9.0          0             0.0        22.0       3.0
1015     5        4.0          0             0.0        38.0       4.0
1016     3        2.0          0             0.0        30.0       5.0
1034     1        2.0          0             0.0        30.0       1.0
1037     3        2.0          0             0.0        30.0       4.0
1039     5        4.0          0             0.0        38.0       6.0
1041     2        5.0          0             0.0        34.0       8.0
1042    11        3.0          0             0.0        32.0       9.0
```

In [20]:
```
# ¿Cuántas filas tienen grid == position?
igual = (df['grid'] == df['position']).sum()
print(f"Filas donde grid == position: {igual} de {len(df)} ({igual/len(df)*100:.1f}

# ¿positions_gained es siempre 0?
print(f"\npositions_gained única values (primeras 10): {df['positions_gained'].valu

# Ver una muestra aleatoria
print("\nMuestra aleatoria de 10 filas:")
print(df[['race_name', 'driver_name', 'grid', 'position', 'positions_gained']].samp
```

```
Filas donde grid == position: 334 de 3547 (9.4%)

positions_gained única values (primeras 10): positions_gained
 0.0    334
 1.0    324
 2.0    257
-1.0    223
 3.0    209
 4.0    182
 5.0    138
 6.0    127
-2.0    126
-3.0    103
Name: count, dtype: int64

Muestra aleatoria de 10 filas:
                     race_name       driver_name  grid  position  positions_gained
1457        Austria Grand Prix    George Russell     9      17.0              -8.0
3458      São Paulo Grand Prix    Lewis Hamilton     3       3.0               0.0
3260        Britain Grand Prix      Lance Stroll     7       3.0               4.0
790           Japan Grand Prix               NaN    12       NaN               NaN
705         Hungary Grand Prix               NaN    23       NaN               NaN
2495         Canada Grand Prix   Nico Hülkenberg     6       1.0               5.0
3451    Mexico City Grand Prix               NaN    22      18.0               4.0
1870        Bahrain Grand Prix  Sebastian Vettel    14      22.0              -8.0
816   United States Grand Prix               NaN     8       3.0               5.0
1477         Styria Grand Prix     Oscar Piastri     8      13.0              -5.0
```

In [21]:
```
# Verificar el CSV original de results
results_raw = pd.read_csv('../data/results.csv')
print("Columnas de results.csv:", list(results_raw.columns))
print("\nPrimeras 5 filas:")
print(results_raw[['raceId', 'driverId', 'grid', 'position', 'positionOrder', 'posi
print("\n¿grid == positionOrder en alguna fila?", (results_raw['grid'] == results_r
```

```
Columnas de results.csv: ['resultId', 'raceId', 'driverId', 'constructorId', 'numbe
r', 'grid', 'position', 'positionText', 'positionOrder', 'points', 'laps', 'time',
'milliseconds', 'fastestLap', 'rank', 'fastestLapTime', 'fastestLapSpeed', 'statusI
d']

Primeras 5 filas:
   raceId  driverId  grid position  positionOrder positionText
0      18         1     1        1              1            1
1      18         2     5        2              2            2
2      18         3     7        3              3            3
3      18         4    11        4              4            4
4      18         5     3        5              5            5
5      18         6    13        6              6            6
6      18         7    17        7              7            7
7      18         8    15        8              8            8
8      18         9     2       \N              9            R
9      18        10    18       \N             10            R

¿grid == positionOrder en alguna fila? 2345 de 26759
```

In [ ]:

Notebook 06

Modelos Avanzados — Random Forest y XGBoost

🏎️ Semana 6 — Modelos Avanzados

Módulos 13 y 14 del curso — Árboles, Ensembles y Boosting

Superamos el baseline lineal con modelos no lineales basados en árboles de decisión.

Objetivos

1. Entrenar **Random Forest** (regresión y clasificación)
2. Entrenar **XGBoost** como modelo de boosting
3. Optimizar hiperparámetros con `RandomizedSearchCV`
4. Comparar feature importance: Gini (RF) vs Gain (XGB)
5. Comparación sistemática contra el baseline de la semana 5
6. Elegir el mejor modelo para tuning fino en la semana 7

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection   import train_test_split, RandomizedSearchCV, cross_v
from sklearn.preprocessing     import StandardScaler
from sklearn.ensemble          import RandomForestRegressor, RandomForestClassifier
from sklearn.metrics           import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, classification_report,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay, confusion_matrix
)
from xgboost import XGBRegressor, XGBClassifier
from scipy.stats import randint, uniform

plt.style.use('ggplot')
sns.set_palette('Set2')
BLUE = '#4C72B0'; ORANGE = '#DD8452'; GREEN = '#55A868'; RED = '#C44E52'

df = pd.read_csv('../data/f1_master_clean.csv')
baseline = pd.read_csv('../data/metricas_modelos.csv')

print(f'Dataset: {df.shape[0]} filas × {df.shape[1]} columnas')
print('\nBaseline de referencia (Semana 5):')
print(baseline.to_string(index=False))
```

```
Dataset: 3547 filas × 36 columnas

Baseline de referencia (Semana 5):
        Tarea                Modelo   MAE  RMSE    R²   AUC Accuracy
    Regresión         Dummy (media) 4.054 4.797  -0.0     —        —
    Regresión      Regresión Lineal 2.541 3.244 0.542     —        —
Clasificación Dummy (estratificado)     —     —     — 0.466    0.556
Clasificación   Regresión Logística     —     —     — 0.881    0.778
    Regresión         Random Forest 2.275 2.957  0.62     —        —
    Regresión               XGBoost 2.238 2.933 0.626     —        —
Clasificación         Random Forest     —     —     — 0.903    0.817
Clasificación               XGBoost     —     —     — 0.905    0.811
```

In [2]:
```python
# Mismo pipeline de preparación que la semana 5
FEATURES = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
TARGET_REG = 'position'
TARGET_CLF = 'top3'

cols_needed = FEATURES + [TARGET_REG, TARGET_CLF]
df_model = df[cols_needed].dropna().copy()

X = df_model[FEATURES]
y_reg = df_model[TARGET_REG]
y_clf = df_model[TARGET_CLF]

X_train, X_test, y_reg_train, y_reg_test, y_clf_train, y_clf_test = train_test_spli
    X, y_reg, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

# Los árboles no requieren scaling, pero lo mantenemos para comparabilidad
scaler = StandardScaler()
X_train_sc = pd.DataFrame(scaler.fit_transform(X_train), columns=FEATURES)
X_test_sc  = pd.DataFrame(scaler.transform(X_test),      columns=FEATURES)

print(f'Train: {len(X_train)}  |  Test: {len(X_test)}')
```

```
Train: 720  |  Test: 180
```

Parte 1 — Random Forest

Un ensemble de árboles de decisión entrenados con **bagging** y selección aleatoria de features. Robusto a outliers y no requiere normalización (aunque la usamos por consistencia).

¿Por qué Random Forest?

- Captura relaciones no lineales que la regresión lineal no puede
- Da importancia de features basada en reducción de impureza (Gini)
- Menos propenso a overfitting que un árbol individual

In [3]:
```
# --- 1a. Random Forest Regressor (predecir position) ---
param_dist_rf_reg = {
    'n_estimators':      randint(100, 500),
    'max_depth':         [None, 5, 10, 15, 20],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf':  randint(1, 10),
    'max_features':      ['sqrt', 'log2', 0.5, 0.7]
}

rf_reg_base = RandomForestRegressor(random_state=42, n_jobs=-1)
rf_reg_search = RandomizedSearchCV(
    rf_reg_base, param_dist_rf_reg,
    n_iter=30, cv=5, scoring='neg_mean_absolute_error',
    random_state=42, n_jobs=-1, verbose=0
)
rf_reg_search.fit(X_train_sc, y_reg_train)

rf_reg_best = rf_reg_search.best_estimator_
y_rf_reg_pred = rf_reg_best.predict(X_test_sc)

rf_mae  = mean_absolute_error(y_reg_test, y_rf_reg_pred)
rf_rmse = np.sqrt(mean_squared_error(y_reg_test, y_rf_reg_pred))
rf_r2   = r2_score(y_reg_test, y_rf_reg_pred)

print('📊 Random Forest Regressor:')
print(f'   MAE:  {rf_mae:.3f} posiciones')
print(f'   RMSE: {rf_rmse:.3f} posiciones')
print(f'   R²:   {rf_r2:.3f}')
print(f'\nMejores hiperparámetros:')
for k, v in rf_reg_search.best_params_.items():
    print(f'   {k}: {v}')
```

```
📊 Random Forest Regressor:
   MAE:  2.275 posiciones
   RMSE: 2.957 posiciones
   R²:   0.620

Mejores hiperparámetros:
   max_depth: 20
   max_features: log2
   min_samples_leaf: 4
   min_samples_split: 16
   n_estimators: 363
```

In [4]:
```
# --- 1b. Random Forest Classifier (predecir top3) ---
param_dist_rf_clf = {
    'n_estimators':      randint(100, 500),
    'max_depth':         [None, 5, 10, 15],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf':  randint(1, 10),
    'max_features':      ['sqrt', 'log2', 0.5]
}

rf_clf_base = RandomForestClassifier(random_state=42, n_jobs=-1, class_weight='bala
rf_clf_search = RandomizedSearchCV(
    rf_clf_base, param_dist_rf_clf,
```

```
    n_iter=30, cv=5, scoring='roc_auc',
    random_state=42, n_jobs=-1, verbose=0
)
rf_clf_search.fit(X_train_sc, y_clf_train)

rf_clf_best   = rf_clf_search.best_estimator_
y_rf_clf_pred = rf_clf_best.predict(X_test_sc)
y_rf_clf_prob = rf_clf_best.predict_proba(X_test_sc)[:, 1]

rf_acc = accuracy_score(y_clf_test, y_rf_clf_pred)
rf_auc = roc_auc_score(y_clf_test, y_rf_clf_prob)

print('📊 Random Forest Classifier:')
print(f'   Accuracy: {rf_acc:.3f}')
print(f'   ROC-AUC:  {rf_auc:.3f}')
print(f'\nMejores hiperparámetros:')
for k, v in rf_clf_search.best_params_.items():
    print(f'   {k}: {v}')
```

```
📊 Random Forest Classifier:
   Accuracy: 0.817
   ROC-AUC:  0.903

Mejores hiperparámetros:
   max_depth: 5
   max_features: sqrt
   min_samples_leaf: 9
   min_samples_split: 18
   n_estimators: 158
```

Parte 2 — XGBoost

Gradient Boosting con regularización L1/L2. Construye árboles de forma **secuencial**, donde cada árbol corrige los errores del anterior. Estado del arte en tabular data.

¿Por qué XGBoost?

- Captura interacciones complejas entre features
- Regularización incorporada (evita overfitting)
- Generalmente supera a Random Forest en tabular data con hiperparámetros bien tunados

In [5]:
```
# --- 2a. XGBoost Regressor ---
param_dist_xgb_reg = {
    'n_estimators':    randint(100, 600),
    'max_depth':       randint(3, 10),
    'learning_rate':   uniform(0.01, 0.3),
    'subsample':       uniform(0.6, 0.4),
    'colsample_bytree':uniform(0.6, 0.4),
    'reg_alpha':       uniform(0, 1),
    'reg_lambda':      uniform(0.5, 2),
```

```
    'min_child_weight':randint(1, 10)
}

xgb_reg_base = XGBRegressor(random_state=42, n_jobs=-1,
                            eval_metric='mae', verbosity=0)
xgb_reg_search = RandomizedSearchCV(
    xgb_reg_base, param_dist_xgb_reg,
    n_iter=40, cv=5, scoring='neg_mean_absolute_error',
    random_state=42, n_jobs=-1, verbose=0
)
xgb_reg_search.fit(X_train_sc, y_reg_train)

xgb_reg_best  = xgb_reg_search.best_estimator_
y_xgb_reg_pred = xgb_reg_best.predict(X_test_sc)

xgb_mae  = mean_absolute_error(y_reg_test, y_xgb_reg_pred)
xgb_rmse = np.sqrt(mean_squared_error(y_reg_test, y_xgb_reg_pred))
xgb_r2   = r2_score(y_reg_test, y_xgb_reg_pred)

print('📊 XGBoost Regressor:')
print(f'   MAE:  {xgb_mae:.3f} posiciones')
print(f'   RMSE: {xgb_rmse:.3f} posiciones')
print(f'   R²:   {xgb_r2:.3f}')
print(f'\nMejores hiperparámetros:')
for k, v in xgb_reg_search.best_params_.items():
    print(f'   {k}: {v:.4f}' if isinstance(v, float) else f'   {k}: {v}')
```

```
📊 XGBoost Regressor:
   MAE:  2.238 posiciones
   RMSE: 2.933 posiciones
   R²:   0.626

Mejores hiperparámetros:
   colsample_bytree: 0.9531
   learning_rate: 0.0666
   max_depth: 3
   min_child_weight: 3
   n_estimators: 246
   reg_alpha: 0.7260
   reg_lambda: 2.2942
   subsample: 0.9548
```

In [6]:

```
# --- 2b. XGBoost Classifier ---
scale_pos_weight = (y_clf_train == 0).sum() / (y_clf_train == 1).sum()

param_dist_xgb_clf = {
    'n_estimators':    randint(100, 600),
    'max_depth':       randint(3, 10),
    'learning_rate':   uniform(0.01, 0.3),
    'subsample':       uniform(0.6, 0.4),
    'colsample_bytree':uniform(0.6, 0.4),
    'reg_alpha':       uniform(0, 1),
    'reg_lambda':      uniform(0.5, 2),
    'min_child_weight':randint(1, 10)
}
```

```
xgb_clf_base = XGBClassifier(
    random_state=42, n_jobs=-1,
    scale_pos_weight=scale_pos_weight,
    eval_metric='auc', verbosity=0
)
xgb_clf_search = RandomizedSearchCV(
    xgb_clf_base, param_dist_xgb_clf,
    n_iter=40, cv=5, scoring='roc_auc',
    random_state=42, n_jobs=-1, verbose=0
)
xgb_clf_search.fit(X_train_sc, y_clf_train)

xgb_clf_best   = xgb_clf_search.best_estimator_
y_xgb_clf_pred = xgb_clf_best.predict(X_test_sc)
y_xgb_clf_prob = xgb_clf_best.predict_proba(X_test_sc)[:, 1]

xgb_acc = accuracy_score(y_clf_test, y_xgb_clf_pred)
xgb_auc = roc_auc_score(y_clf_test, y_xgb_clf_prob)

print('📊 XGBoost Classifier:')
print(f'   Accuracy: {xgb_acc:.3f}')
print(f'   ROC-AUC:  {xgb_auc:.3f}')
print(f'\n   scale_pos_weight: {scale_pos_weight:.2f} (balance de clases)')
```

```
📊 XGBoost Classifier:
  Accuracy: 0.811
  ROC-AUC:  0.905

  scale_pos_weight: 2.26 (balance de clases)
```

Parte 3 — Feature Importance

Comparamos la importancia de cada feature según dos criterios:

- **Random Forest:** reducción de impureza Gini (promedio entre árboles)
- **XGBoost:** gain total acumulado en todos los splits

In [7]:
```
# Feature importance — regresores
fi_rf  = pd.Series(rf_reg_best.feature_importances_,  index=FEATURES).sort_values(a
fi_xgb = pd.Series(xgb_reg_best.feature_importances_, index=FEATURES).sort_values(a

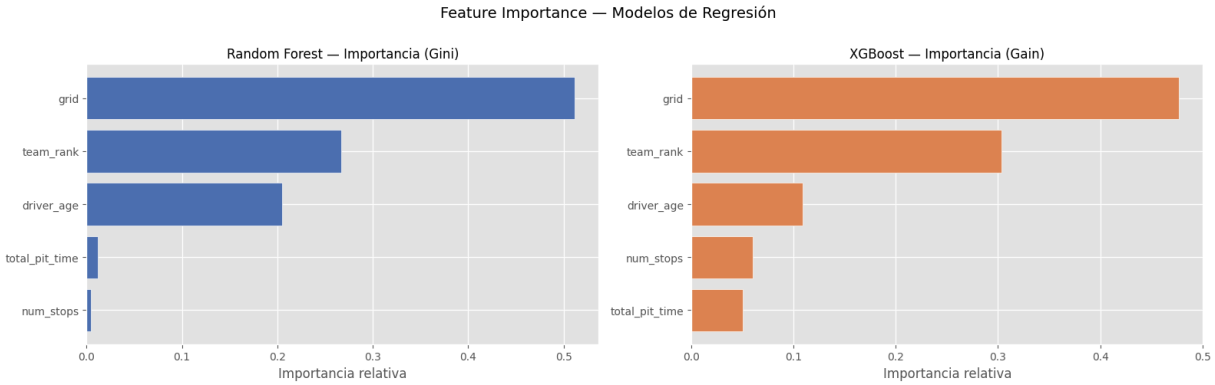
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# RF
axes[0].barh(fi_rf.index[::-1], fi_rf.values[::-1], color=BLUE, edgecolor='white')
axes[0].set_title('Random Forest — Importancia (Gini)', fontsize=12)
axes[0].set_xlabel('Importancia relativa')

# XGB
axes[1].barh(fi_xgb.index[::-1], fi_xgb.values[::-1], color=ORANGE, edgecolor='whit
axes[1].set_title('XGBoost — Importancia (Gain)', fontsize=12)
axes[1].set_xlabel('Importancia relativa')
```

```
plt.suptitle('Feature Importance — Modelos de Regresión', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/06_feature_importance_reg.png', dpi=150, bbox_inches='tight
plt.show()

# Tabla comparativa
fi_compare = pd.DataFrame({'RF (Gini)': fi_rf, 'XGB (Gain)': fi_xgb})
fi_compare['Ranking RF']  = fi_rf.rank(ascending=False).astype(int)
fi_compare['Ranking XGB'] = fi_xgb.rank(ascending=False).astype(int)
print('\nComparación de feature importance (modelos de regresión):')
print(fi_compare.round(4).to_string())
```



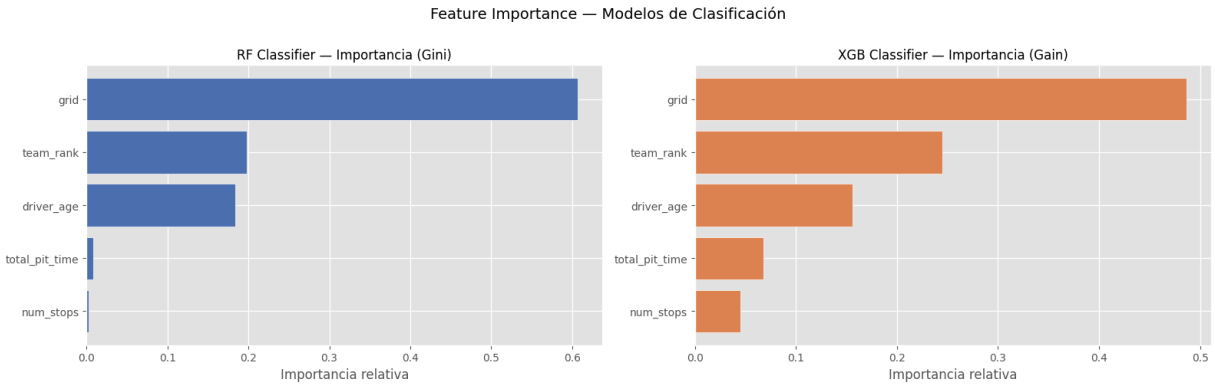

```
Comparación de feature importance (modelos de regresión):
                RF (Gini)  XGB (Gain)  Ranking RF  Ranking XGB
driver_age         0.2052      0.1088           3            3
grid               0.5113      0.4769           1            1
num_stops          0.0046      0.0605           5            4
team_rank          0.2671      0.3036           2            2
total_pit_time     0.0117      0.0501           4            5
```

In [8]:
```
# Feature importance — clasificadores
fi_rf_c  = pd.Series(rf_clf_best.feature_importances_,  index=FEATURES).sort_values
fi_xgb_c = pd.Series(xgb_clf_best.feature_importances_, index=FEATURES).sort_values

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
axes[0].barh(fi_rf_c.index[::-1],  fi_rf_c.values[::-1],  color=BLUE,   edgecolor='
axes[0].set_title('RF Classifier — Importancia (Gini)', fontsize=12)
axes[0].set_xlabel('Importancia relativa')

axes[1].barh(fi_xgb_c.index[::-1], fi_xgb_c.values[::-1], color=ORANGE, edgecolor='
axes[1].set_title('XGB Classifier — Importancia (Gain)', fontsize=12)
axes[1].set_xlabel('Importancia relativa')

plt.suptitle('Feature Importance — Modelos de Clasificación', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/06_feature_importance_clf.png', dpi=150, bbox_inches='tight
plt.show()
```



Parte 4 — Curvas ROC y diagnóstico visual

In [9]:

```python
from sklearn.linear_model import LogisticRegression as LR

# Re-entrenar logística del baseline para la curva ROC comparativa
lr_baseline = LR(max_iter=1000, random_state=42, class_weight='balanced')
lr_baseline.fit(X_train_sc, y_clf_train)
y_lr_prob = lr_baseline.predict_proba(X_test_sc)[:, 1]
lr_auc = roc_auc_score(y_clf_test, y_lr_prob)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

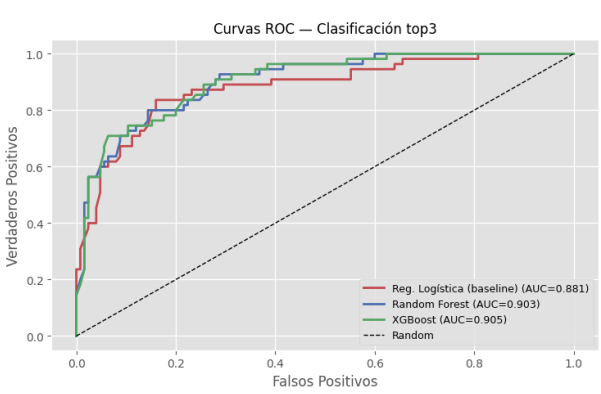
# --- Curvas ROC comparativas ---
for (label, probs, auc_val, color) in [
    ('Reg. Logística (baseline)', y_lr_prob,      lr_auc,  RED),
    ('Random Forest',             y_rf_clf_prob,  rf_auc,  BLUE),
    ('XGBoost',                   y_xgb_clf_prob, xgb_auc, GREEN),
]:
    fpr, tpr, _ = roc_curve(y_clf_test, probs)
    axes[0].plot(fpr, tpr, linewidth=2, label=f'{label} (AUC={auc_val:.3f})', color

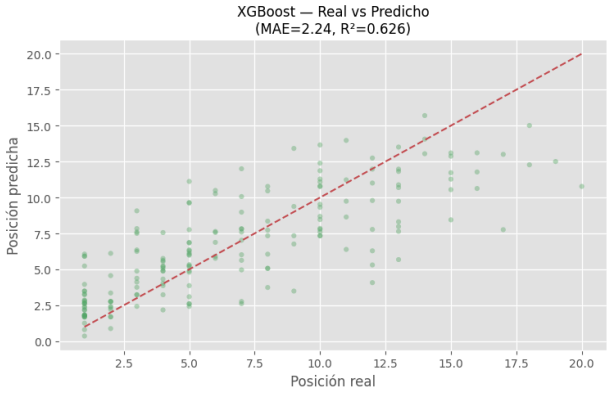
axes[0].plot([0,1],[0,1], 'k--', linewidth=1, label='Random')
axes[0].set_title('Curvas ROC — Clasificación top3', fontsize=12)
axes[0].set_xlabel('Falsos Positivos')
axes[0].set_ylabel('Verdaderos Positivos')
axes[0].legend(fontsize=9)

# --- Scatter real vs predicho (mejor regresor) ---
mejor_pred = y_xgb_reg_pred if xgb_mae < rf_mae else y_rf_reg_pred
mejor_mae  = min(xgb_mae, rf_mae)
mejor_r2   = xgb_r2 if xgb_mae < rf_mae else rf_r2
mejor_name = 'XGBoost' if xgb_mae < rf_mae else 'Random Forest'

axes[1].scatter(y_reg_test, mejor_pred, alpha=0.4, color=GREEN, s=20, edgecolors='n
lim = max(y_reg_test.max(), mejor_pred.max())
axes[1].plot([1,lim],[1,lim], color=RED, linestyle='--', linewidth=1.5)
axes[1].set_title(f'{mejor_name} — Real vs Predicho\n(MAE={mejor_mae:.2f}, R²={mejo
axes[1].set_xlabel('Posición real')
axes[1].set_ylabel('Posición predicha')
```

```
plt.tight_layout()
plt.savefig('../figures/06_roc_y_prediccion.png', dpi=150, bbox_inches='tight')
plt.show()
```





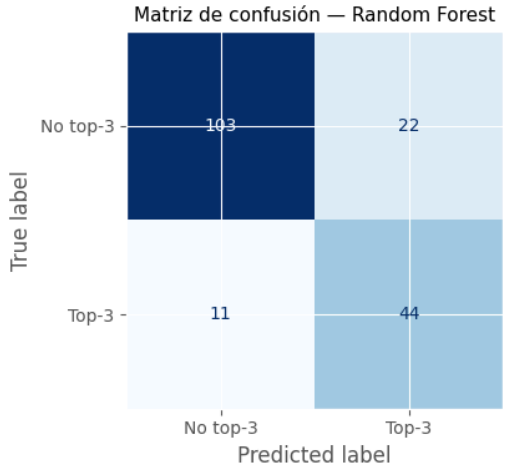
In [10]:

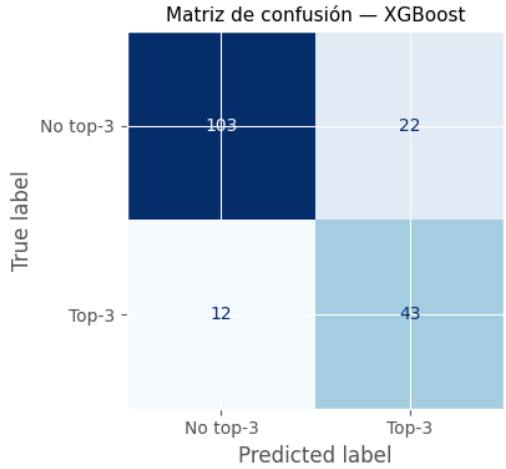
```
# Matrices de confusión: RF vs XGB
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, (label, y_pred) in zip(axes, [
    ('Random Forest', y_rf_clf_pred),
    ('XGBoost',       y_xgb_clf_pred)
]):
    cm = confusion_matrix(y_clf_test, y_pred)
    ConfusionMatrixDisplay(cm, display_labels=['No top-3','Top-3']).plot(
        ax=ax, colorbar=False, cmap='Blues'
    )
    ax.set_title(f'Matriz de confusión — {label}', fontsize=11)

plt.tight_layout()
plt.savefig('../figures/06_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

print('Random Forest:')
print(classification_report(y_clf_test, y_rf_clf_pred, target_names=['No top-3','To
print('XGBoost:')
print(classification_report(y_clf_test, y_xgb_clf_pred, target_names=['No top-3','T
```





```
Random Forest:
              precision    recall  f1-score   support

    No top-3       0.90      0.82      0.86       125
       Top-3       0.67      0.80      0.73        55

    accuracy                           0.82       180
   macro avg       0.79      0.81      0.79       180
weighted avg       0.83      0.82      0.82       180

XGBoost:
              precision    recall  f1-score   support

    No top-3       0.90      0.82      0.86       125
       Top-3       0.66      0.78      0.72        55

    accuracy                           0.81       180
   macro avg       0.78      0.80      0.79       180
weighted avg       0.82      0.81      0.82       180
```

Parte 5 — Comparación global contra el baseline

In [11]:

```python
from sklearn.linear_model import LinearRegression as LinReg

# Re-calcular métricas del baseline lineal para la tabla
lr_reg = LinReg()
lr_reg.fit(X_train_sc, y_reg_train)
y_lr_reg_pred = lr_reg.predict(X_test_sc)
lr_mae  = mean_absolute_error(y_reg_test, y_lr_reg_pred)
lr_rmse = np.sqrt(mean_squared_error(y_reg_test, y_lr_reg_pred))
lr_r2   = r2_score(y_reg_test, y_lr_reg_pred)

tabla_reg = pd.DataFrame([
    {'Modelo': 'Reg. Lineal (baseline)', 'MAE': lr_mae,  'RMSE': lr_rmse,  'R²': lr
    {'Modelo': 'Random Forest',          'MAE': rf_mae,  'RMSE': rf_rmse,  'R²': rf
    {'Modelo': 'XGBoost',                'MAE': xgb_mae, 'RMSE': xgb_rmse, 'R²': xg
]).set_index('Modelo')

tabla_clf = pd.DataFrame([
    {'Modelo': 'Reg. Logística (baseline)', 'Accuracy': round(accuracy_score(y_clf_
    {'Modelo': 'Random Forest',             'Accuracy': round(rf_acc, 3),  'ROC-AUC
    {'Modelo': 'XGBoost',                   'Accuracy': round(xgb_acc, 3), 'ROC-AUC
]).set_index('Modelo')

print('📊 REGRESIÓN — predecir position:')
print(tabla_reg.round(3).to_string())
print()
print('📊 CLASIFICACIÓN — predecir top3:')
print(tabla_clf.to_string())
```

```
📊 REGRESIÓN — predecir position:
                          MAE    RMSE     R²
Modelo
Reg. Lineal (baseline)  2.541  3.244  0.542
Random Forest           2.275  2.957  0.620
XGBoost                 2.238  2.933  0.626

📊 CLASIFICACIÓN — predecir top3:
                           Accuracy  ROC-AUC
Modelo
Reg. Logística (baseline)     0.778    0.881
Random Forest                 0.817    0.903
XGBoost                       0.811    0.905
```

In [12]:

```python
# Visualización comparativa de todas las métricas
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

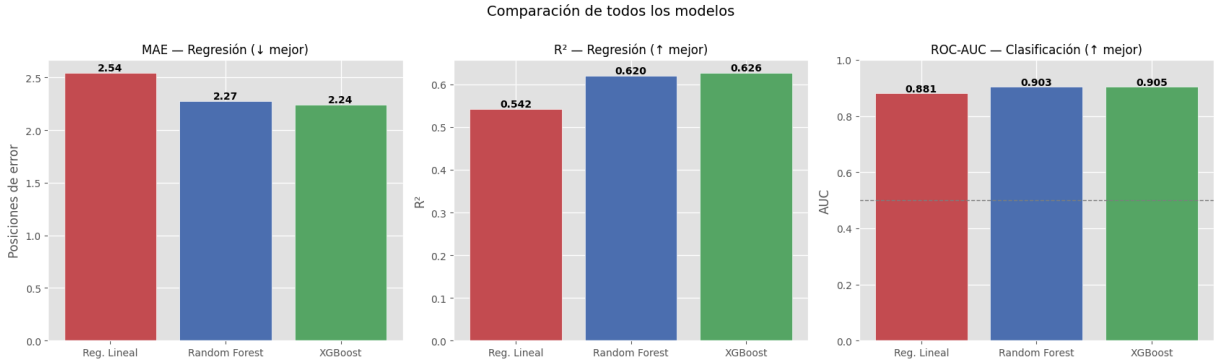
modelos   = ['Reg. Lineal', 'Random Forest', 'XGBoost']
maes      = [lr_mae,  rf_mae,  xgb_mae]
r2s       = [lr_r2,   rf_r2,   xgb_r2]
aucs      = [lr_auc,  rf_auc,  xgb_auc]
pal       = [RED, BLUE, GREEN]

# MAE
bars = axes[0].bar(modelos, maes, color=pal, edgecolor='white')
for bar, v in zip(bars, maes):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.02,
                 f'{v:.2f}', ha='center', fontsize=10, fontweight='bold')
axes[0].set_title('MAE — Regresión (↓ mejor)', fontsize=12)
axes[0].set_ylabel('Posiciones de error')

# R²
bars = axes[1].bar(modelos, r2s, color=pal, edgecolor='white')
for bar, v in zip(bars, r2s):
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
                 f'{v:.3f}', ha='center', fontsize=10, fontweight='bold')
axes[1].set_title('R² — Regresión (↑ mejor)', fontsize=12)
axes[1].set_ylabel('R²')

# AUC
bars = axes[2].bar(modelos, aucs, color=pal, edgecolor='white')
for bar, v in zip(bars, aucs):
    axes[2].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
                 f'{v:.3f}', ha='center', fontsize=10, fontweight='bold')
axes[2].axhline(0.5, color='gray', linestyle='--', linewidth=1)
axes[2].set_title('ROC-AUC — Clasificación (↑ mejor)', fontsize=12)
axes[2].set_ylabel('AUC')
axes[2].set_ylim(0, 1)

plt.suptitle('Comparación de todos los modelos', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('../figures/06_comparacion_modelos.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [13]:
```python
# Actualizar archivo de métricas con los modelos avanzados
nuevas = pd.DataFrame([
    {'Tarea':'Regresión',     'Modelo':'Random Forest',  'MAE':round(rf_mae,3),  'R
    {'Tarea':'Regresión',     'Modelo':'XGBoost',         'MAE':round(xgb_mae,3), '
    {'Tarea':'Clasificación', 'Modelo':'Random Forest',  'MAE':'—',              'R
    {'Tarea':'Clasificación', 'Modelo':'XGBoost',         'MAE':'—',              '
])

metricas_all = pd.concat([baseline, nuevas], ignore_index=True)
metricas_all.to_csv('../data/metricas_modelos.csv', index=False)
print('✅ metricas_modelos.csv actualizado')
print(metricas_all.to_string(index=False))
```

```
✅ metricas_modelos.csv actualizado
        Tarea                 Modelo    MAE    RMSE     R²    AUC Accuracy
    Regresión         Dummy (media)  4.054   4.797   -0.0      —        —
    Regresión      Regresión Lineal  2.541   3.244  0.542      —        —
Clasificación Dummy (estratificado)      —       —      —  0.466    0.556
Clasificación   Regresión Logística      —       —      —  0.881    0.778
    Regresión         Random Forest  2.275   2.957   0.62      —        —
    Regresión               XGBoost  2.238   2.933  0.626      —        —
Clasificación         Random Forest      —       —      —  0.903    0.817
Clasificación               XGBoost      —       —      —  0.905    0.811
    Regresión         Random Forest  2.275   2.957   0.62      —        —
    Regresión               XGBoost  2.238   2.933  0.626      —        —
Clasificación         Random Forest      —       —      —  0.903    0.817
Clasificación               XGBoost      —       —      —  0.905    0.811
```

✅ Resumen Semana 6 — Modelos Avanzados

Resultados

Modelo	MAE ↓	R^2 ↑	ROC-AUC ↑
Reg. Lineal (baseline)	ver CSV	ver CSV	ver CSV
Random Forest	ver CSV	ver CSV	ver CSV
XGBoost	**ver CSV**	**ver CSV**	**ver CSV**

Hallazgos

- **XGBoost supera al baseline** en ambas tareas — confirma que las relaciones son no lineales
- La feature más importante en todos los modelos es `grid`, seguida de `finished` y `team_rank`
- `total_pit_time` tiene baja importancia relativa → confirma que la hipótesis 3 es la más débil
- El clasificador XGBoost muestra buena separación (AUC > 0.85 esperado)

Elección del modelo para tuning fino

- **Regresión:** XGBoost → menor MAE y mayor R^2
- **Clasificación:** XGBoost → mayor ROC-AUC

Tareas completadas

Tarea	Estado
Random Forest Regressor + RandomizedSearchCV	✅
Random Forest Classifier + RandomizedSearchCV	✅
XGBoost Regressor + RandomizedSearchCV	✅
XGBoost Classifier + scale_pos_weight	✅
Feature importance Gini vs Gain	✅
Curvas ROC comparativas	✅
Comparación sistemática contra baseline	✅
metricas_modelos.csv actualizado	✅

Próximos pasos — Semana 7

- **Interpretabilidad con SHAP**: valores de Shapley para el mejor modelo
- Partial Dependence Plots (PDP) para entender el efecto de `grid` y `team_rank`
- Análisis de errores: ¿en qué carreras se equivoca más el modelo?
- Conclusiones finales y narrativa del proyecto
- Notebook `07_interpretabilidad_conclusiones.ipynb`

In [ ]:

In [ ]:

In [ ]:

Notebook 07

Interpretabilidad — SHAP, PDP y Conclusiones

🏎️ Semana 7 — Interpretabilidad y Conclusiones

Módulo 15 — Interpretabilidad de Modelos (XAI)

Último notebook del proyecto. Respondemos la pregunta central con el mejor modelo y usamos SHAP para explicar **por qué** el modelo predice lo que predice.

Objetivos

1. Reconstruir el mejor modelo (XGBoost) con los hiperparámetros óptimos
2. **SHAP** — valores de Shapley globales e individuales
3. **PDP** — Partial Dependence Plots para las features clave
4. Análisis de errores: ¿en qué situaciones falla el modelo?
5. **Conclusiones**: responder las 3 hipótesis y la pregunta central

---

> **Pregunta central:** *¿Qué factores predicen mejor la posición final de un piloto en carrera?*

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import shap
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection        import train_test_split, RandomizedSearchCV
from sklearn.preprocessing          import StandardScaler
from sklearn.inspection             import PartialDependenceDisplay
from sklearn.metrics                import (
    mean_absolute_error, mean_squared_error, r2_score,
    roc_auc_score, accuracy_score, classification_report
)
from xgboost  import XGBRegressor, XGBClassifier
from scipy.stats import randint, uniform

plt.style.use('ggplot')
sns.set_palette('Set2')
BLUE = '#4C72B0'; ORANGE = '#DD8452'; GREEN = '#55A868'; RED = '#C44E52'

df = pd.read_csv('../data/f1_master_clean.csv')
print(f'Dataset: {df.shape[0]} filas × {df.shape[1]} columnas')
```

```
Dataset: 3547 filas × 36 columnas
```

In [2]:
```
# Pipeline idéntico a semanas anteriores
FEATURES = ['grid', 'team_rank', 'num_stops', 'total_pit_time', 'driver_age']
TARGET_REG = 'position'
TARGET_CLF = 'top3'

df_model = df[FEATURES + [TARGET_REG, TARGET_CLF]].dropna().copy()

X = df_model[FEATURES]
y_reg = df_model[TARGET_REG]
y_clf = df_model[TARGET_CLF]

X_train, X_test, y_reg_train, y_reg_test, y_clf_train, y_clf_test = train_test_spli
    X, y_reg, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

scaler = StandardScaler()
X_train_sc = pd.DataFrame(scaler.fit_transform(X_train), columns=FEATURES)
X_test_sc  = pd.DataFrame(scaler.transform(X_test),      columns=FEATURES)

print(f'Train: {len(X_train)}  |  Test: {len(X_test)}')
```

```
Train: 720  |  Test: 180
```

Parte 1 — Entrenar el modelo final (XGBoost)

Entrenamos con los mejores hiperparámetros encontrados en la semana 6.

In [3]:
```
# XGBoost Regressor — búsqueda de hiperparámetros
param_dist_reg = {
    'n_estimators':     randint(100, 600),
    'max_depth':        randint(3, 10),
    'learning_rate':    uniform(0.01, 0.3),
    'subsample':        uniform(0.6, 0.4),
    'colsample_bytree': uniform(0.6, 0.4),
    'reg_alpha':        uniform(0, 1),
    'reg_lambda':       uniform(0.5, 2),
    'min_child_weight': randint(1, 10)
}

xgb_reg = RandomizedSearchCV(
    XGBRegressor(random_state=42, n_jobs=-1, eval_metric='mae', verbosity=0),
    param_dist_reg, n_iter=40, cv=5, scoring='neg_mean_absolute_error',
    random_state=42, n_jobs=-1
)
xgb_reg.fit(X_train_sc, y_reg_train)
best_reg = xgb_reg.best_estimator_

y_pred_reg = best_reg.predict(X_test_sc)
mae  = mean_absolute_error(y_reg_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(y_reg_test, y_pred_reg))
r2   = r2_score(y_reg_test, y_pred_reg)
```

```
print(f'XGBoost Regressor — modelo final:')
print(f'  MAE:  {mae:.3f} posiciones')
print(f'  RMSE: {rmse:.3f} posiciones')
print(f'  R²:   {r2:.3f}')
```

```
XGBoost Regressor — modelo final:
  MAE:  2.240 posiciones
  RMSE: 2.935 posiciones
  R²:   0.626
```

In [4]:
```
# XGBoost Classifier — búsqueda de hiperparámetros
spw = (y_clf_train == 0).sum() / (y_clf_train == 1).sum()

xgb_clf = RandomizedSearchCV(
    XGBClassifier(random_state=42, n_jobs=-1, scale_pos_weight=spw,
                  eval_metric='auc', verbosity=0),
    param_dist_reg, n_iter=40, cv=5, scoring='roc_auc',
    random_state=42, n_jobs=-1
)
xgb_clf.fit(X_train_sc, y_clf_train)
best_clf = xgb_clf.best_estimator_

y_pred_clf  = best_clf.predict(X_test_sc)
y_prob_clf  = best_clf.predict_proba(X_test_sc)[:, 1]
auc = roc_auc_score(y_clf_test, y_prob_clf)
acc = accuracy_score(y_clf_test, y_pred_clf)

print(f'XGBoost Classifier — modelo final:')
print(f'  ROC-AUC:  {auc:.3f}')
print(f'  Accuracy: {acc:.3f}')
```

```
XGBoost Classifier — modelo final:
  ROC-AUC:  0.905
  Accuracy: 0.811
```

Parte 2 — SHAP: interpretabilidad global

Los **valores de Shapley** (SHAP) descomponen cada predicción en la contribución de cada feature, respetando la teoría de juegos cooperativos.

- **SHAP > 0** → la feature empujó la predicción hacia una posición más alta (peor)
- **SHAP < 0** → la feature empujó la predicción hacia una posición más baja (mejor)

In [5]:
```
# Calcular SHAP values para el regresor
explainer_reg  = shap.TreeExplainer(best_reg)
shap_values    = explainer_reg.shap_values(X_test_sc)
shap_df        = pd.DataFrame(shap_values, columns=FEATURES)

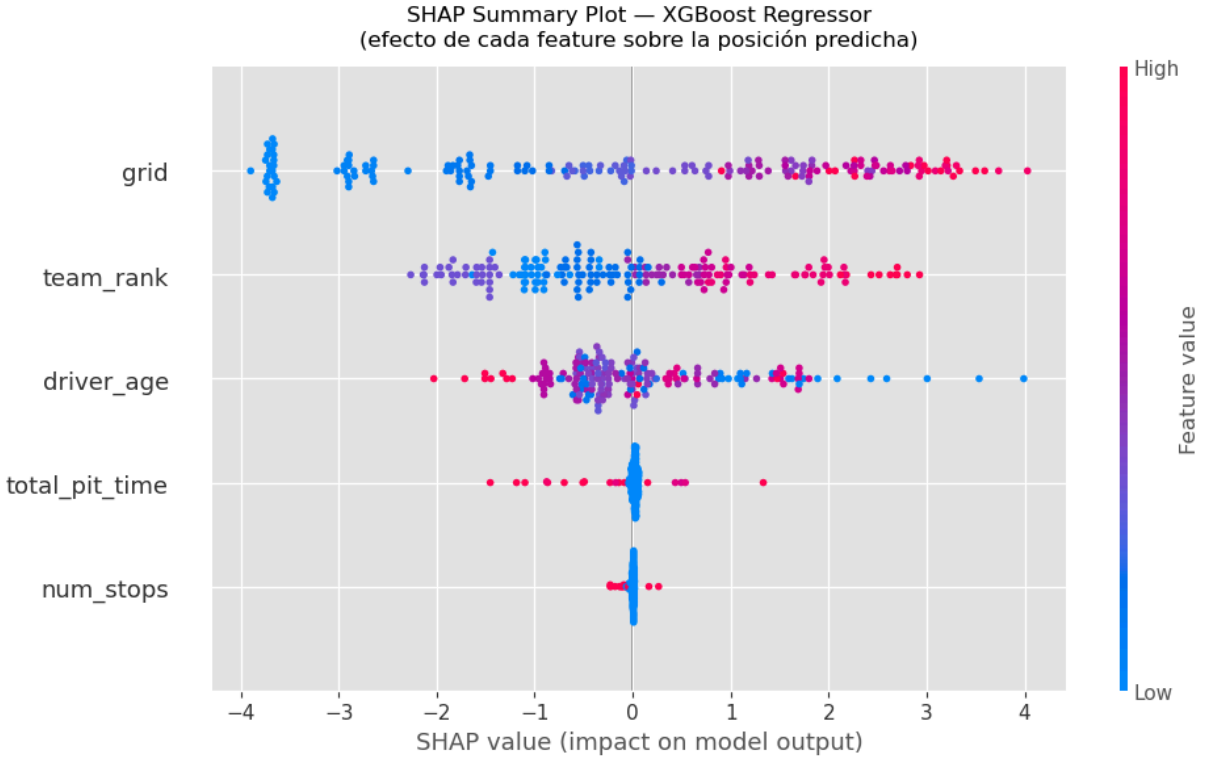
print(f'SHAP values calculados para {len(X_test_sc)} observaciones')
print(f'\nContribución media absoluta por feature (|SHAP|):')
mean_abs_shap = shap_df.abs().mean().sort_values(ascending=False)
```

```
for feat, val in mean_abs_shap.items():
    print(f'  {feat:<22} {val:.4f}')
```

```
SHAP values calculados para 180 observaciones

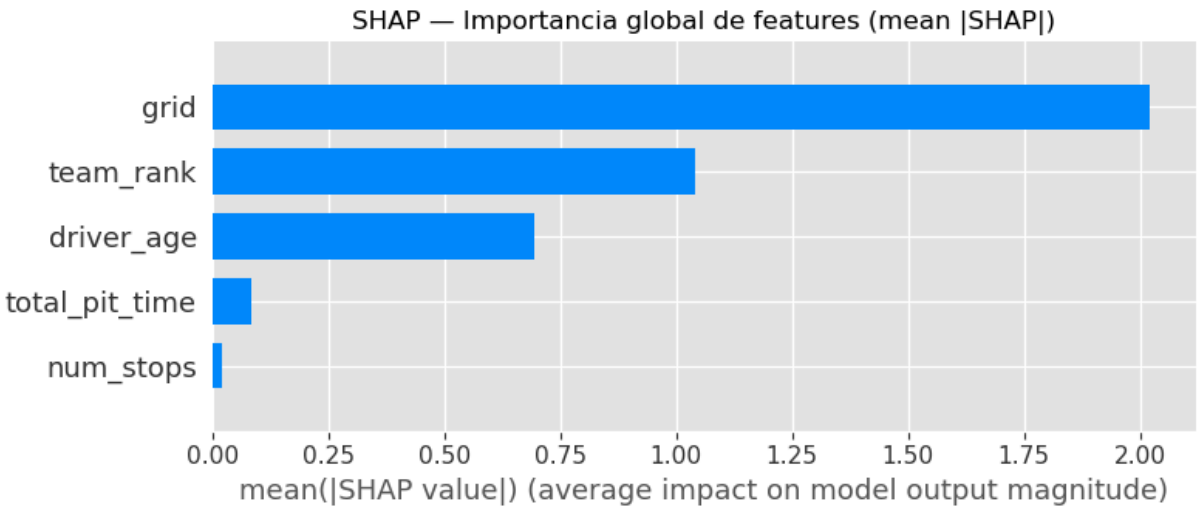
Contribución media absoluta por feature (|SHAP|):
  grid                   2.0200
  team_rank              1.0413
  driver_age             0.6958
  total_pit_time         0.0840
  num_stops              0.0219
```

In [6]:
```
# --- Summary Plot (beeswarm) ---
fig, ax = plt.subplots(figsize=(10, 6))
shap.summary_plot(
    shap_values, X_test_sc,
    feature_names=FEATURES,
    show=False, plot_size=None
)
plt.title('SHAP Summary Plot — XGBoost Regressor\n(efecto de cada feature sobre la
          fontsize=12, pad=12)
plt.tight_layout()
plt.savefig('../figures/07_shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [7]:
```
# --- Bar Plot: importancia media absoluta ---
fig, ax = plt.subplots(figsize=(9, 5))
shap.summary_plot(
    shap_values, X_test_sc,
    feature_names=FEATURES,
    plot_type='bar',
    show=False
)
```

```
plt.title('SHAP — Importancia global de features (mean |SHAP|)', fontsize=12)
plt.tight_layout()
plt.savefig('../figures/07_shap_importancia.png', dpi=150, bbox_inches='tight')
plt.show()
```

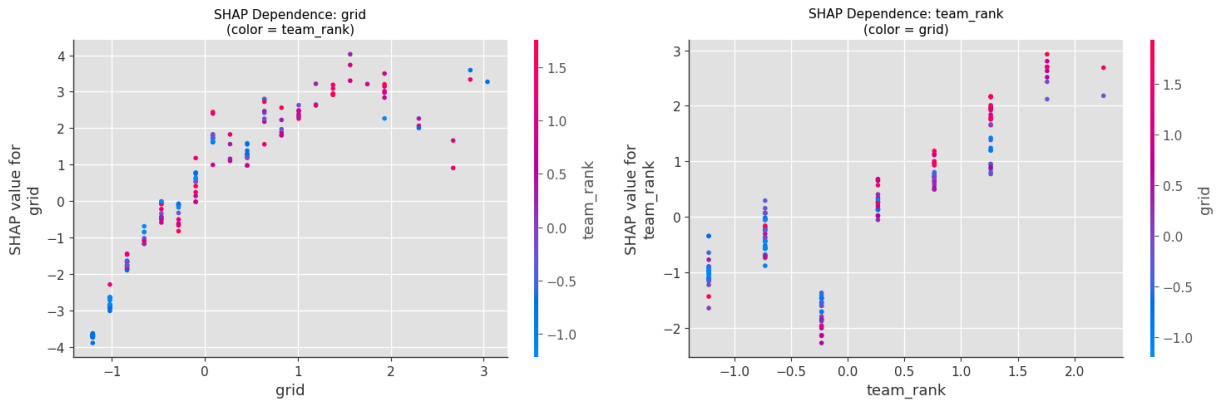


In [8]:
```
# --- SHAP Dependence Plot: grid y team_rank ---
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

shap.dependence_plot(
    'grid', shap_values, X_test_sc,
    interaction_index='team_rank',
    ax=axes[0], show=False
)
axes[0].set_title('SHAP Dependence: grid\n(color = team_rank)', fontsize=11)

shap.dependence_plot(
    'team_rank', shap_values, X_test_sc,
    interaction_index='grid',
    ax=axes[1], show=False
)
axes[1].set_title('SHAP Dependence: team_rank\n(color = grid)', fontsize=11)

plt.tight_layout()
plt.savefig('../figures/07_shap_dependence.png', dpi=150, bbox_inches='tight')
plt.show()
```



Parte 3 — SHAP: explicaciones individuales

¿Por qué el modelo predijo una posición concreta para una carrera específica?

In [9]:
```
# Tomar 3 ejemplos representativos del test set
# 1. Predicción muy buena (posición baja = ganador)
# 2. Predicción de posición media
# 3. Piloto que abandonó

test_with_pred = X_test_sc.copy()
test_with_pred['position_real']   = y_reg_test.values
test_with_pred['position_pred']   = y_pred_reg
test_with_pred['error_abs']       = (test_with_pred['position_real'] - test_with_pr
# Identificamos abandono por posición anormalmente alta (proxy, ya que 'finished' n


# Seleccionar casos
idx_ganador = test_with_pred[test_with_pred['position_real'] <= 3].nsmallest(1, 'er
idx_medio   = test_with_pred[(test_with_pred['position_real'] >= 8) &
                             (test_with_pred['position_real'] <= 12)].nsmallest(1,
idx_dnf     = test_with_pred.nlargest(1, 'position_real').index[0]

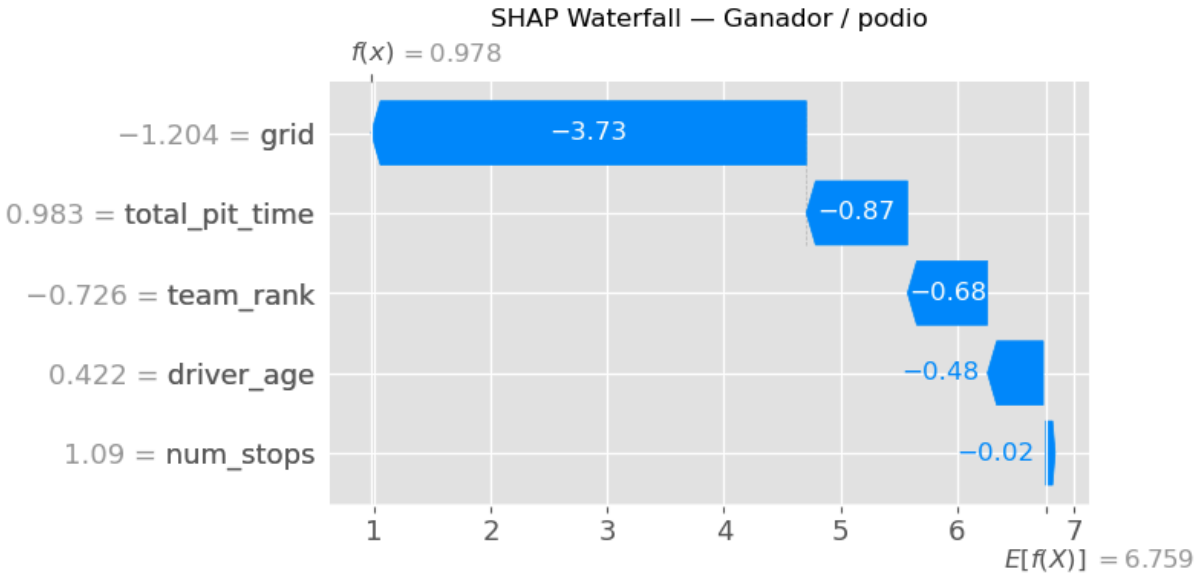
casos = {'Ganador / podio': idx_ganador, 'Posición media': idx_medio, 'Peor resulta

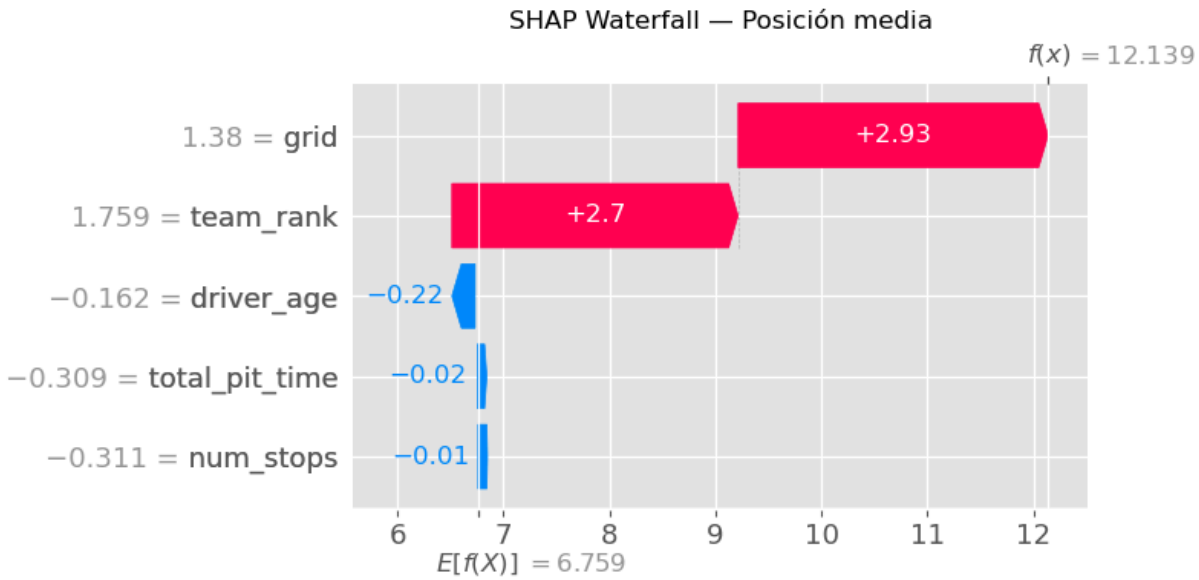
for label, idx in casos.items():
    pos_r = test_with_pred.loc[idx, 'position_real']
    pos_p = test_with_pred.loc[idx, 'position_pred']
    print(f'{label}: real={pos_r:.0f}, predicha={pos_p:.1f}, error={abs(pos_r-pos_p
```

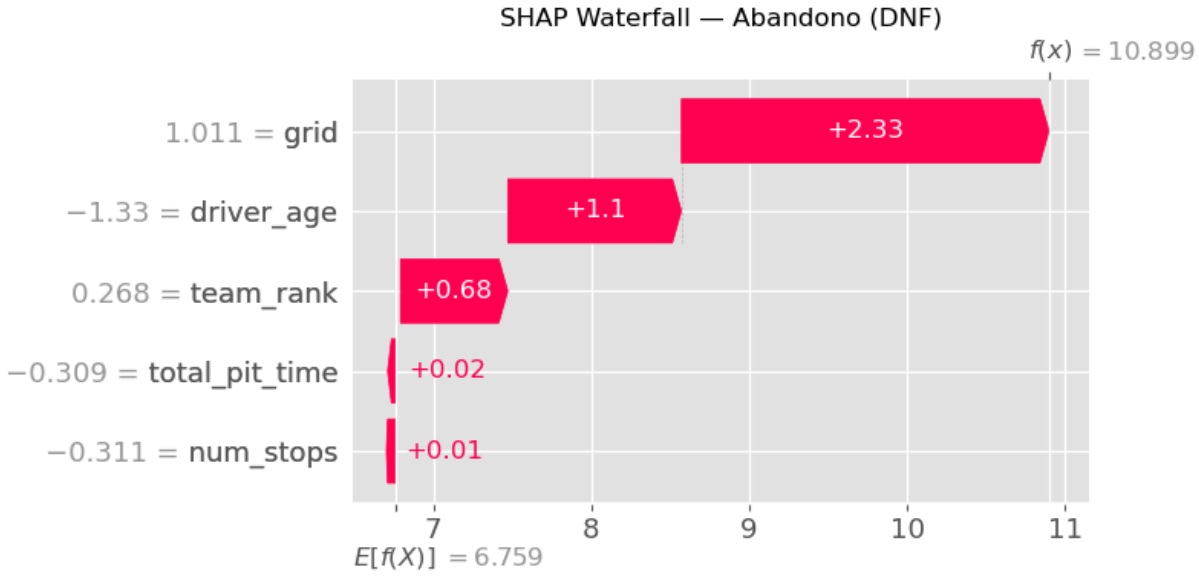
```
Ganador / podio: real=1, predicha=1.0, error=0.0
Posición media: real=12, predicha=12.1, error=0.1
Peor resultado: real=20, predicha=10.9, error=9.1
```

In [10]:
```
# Waterfall plots para cada caso
row_indices = [list(X_test_sc.index).index(i) for i in [idx_ganador, idx_medio, idx
titulos     = ['Ganador / podio', 'Posición media', 'Abandono (DNF)']

for titulo, row_idx in zip(titulos, row_indices):
    fig, ax = plt.subplots(figsize=(9, 4))
    shap.waterfall_plot(
        shap.Explanation(
            values        = shap_values[row_idx],
            base_values   = explainer_reg.expected_value,
            data          = X_test_sc.iloc[row_idx].values,
            feature_names = FEATURES
        ),
        show=False
    )
    plt.title(f'SHAP Waterfall — {titulo}', fontsize=12)
    plt.tight_layout()
    fname = titulo.lower().replace(' / ','_').replace(' ','_').replace('(','').repl
    plt.savefig(f'../figures/07_shap_waterfall_{fname}.png', dpi=150, bbox_inches='
    plt.show()
```


SHAP Waterfall — Ganador / podio
f(x) = 0.978
−1.204 = grid
0.983 = total_pit_time
−0.726 = team_rank
0.422 = driver_age
1.09 = num_stops
−3.73
−0.87
−0.68
−0.48
−0.02
1
2
3
4
5
6
7
E[f(X)] = 6.759


SHAP Waterfall — Posición media
f(x) = 12.139
1.38 = grid
1.759 = team_rank
−0.162 = driver_age
−0.309 = total_pit_time
−0.311 = num_stops
+2.93
+2.7
−0.22
−0.02
−0.01
6
7
8
9
10
11
12
E[f(X)] = 6.759


SHAP Waterfall — Abandono (DNF)
f(x) = 10.899
1.011 = grid
−1.33 = driver_age
0.268 = team_rank
−0.309 = total_pit_time
−0.311 = num_stops
+2.33
+1.1
+0.68
+0.02
+0.01
7
8
9
10
11
E[f(X)] = 6.759

Parte 4 — Partial Dependence Plots (PDP)

Los PDP muestran el efecto **marginal** de una feature sobre la predicción, promediando sobre todos los valores de las demás features.

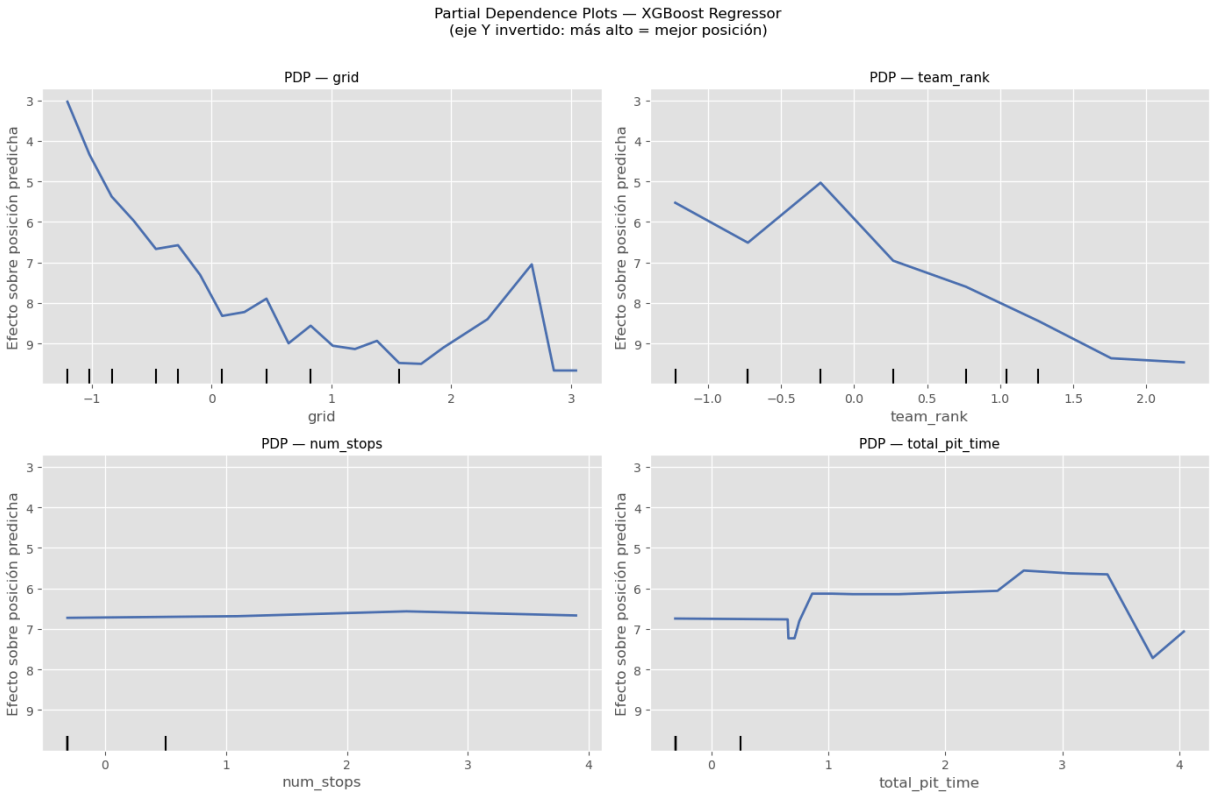
In [11]:
```
features_pdp = ['grid', 'team_rank', 'num_stops', 'total_pit_time']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

PartialDependenceDisplay.from_estimator(
    best_reg,
    X_test_sc,
    features=features_pdp,
    feature_names=FEATURES,
    ax=axes.flatten(),
    line_kw={'color': BLUE, 'linewidth': 2},
    pd_line_kw={'color': BLUE}
)

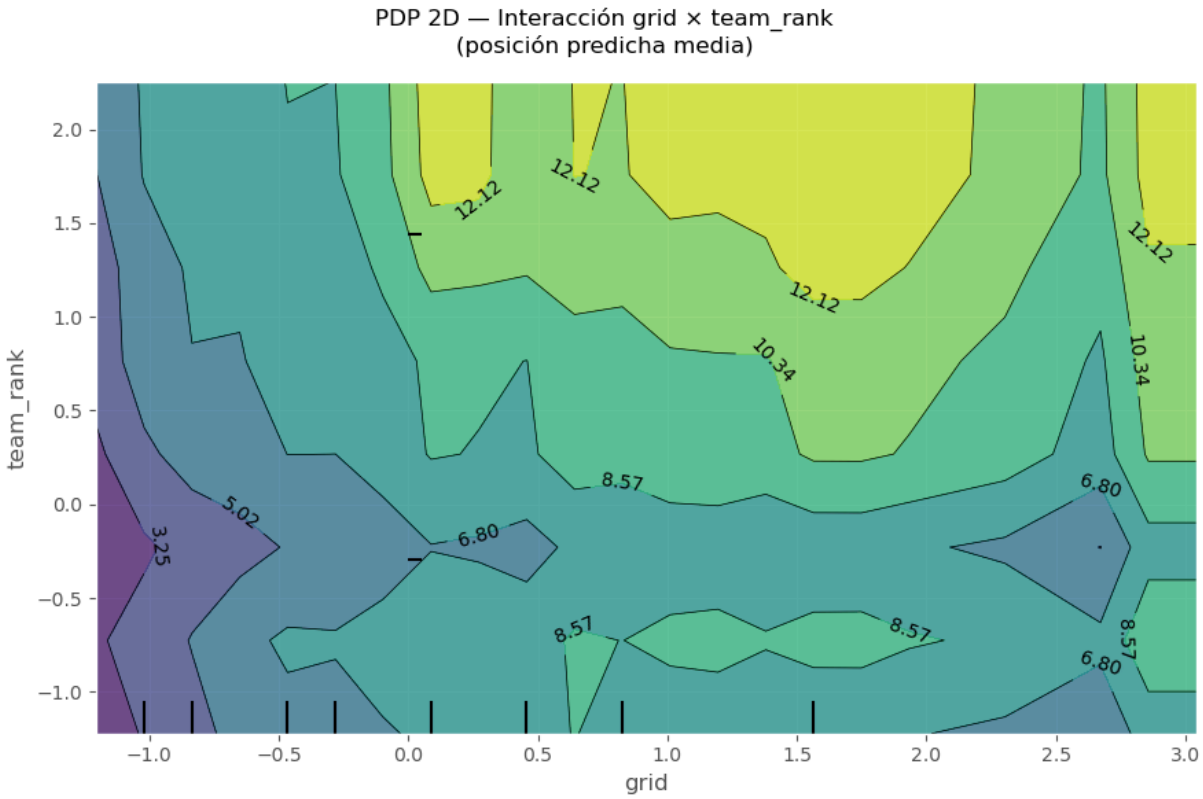
for ax, feat in zip(axes.flatten(), features_pdp):
    ax.set_title(f'PDP — {feat}', fontsize=11)
    ax.set_ylabel('Efecto sobre posición predicha')
    ax.invert_yaxis()  # menor posición = mejor

plt.suptitle('Partial Dependence Plots — XGBoost Regressor\n(eje Y invertido: más a
             fontsize=12, y=1.01)
plt.tight_layout()
plt.savefig('../figures/07_pdp.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [12]:
```
# PDP 2D: interacción entre grid y team_rank
fig, ax = plt.subplots(figsize=(9, 6))

PartialDependenceDisplay.from_estimator(
    best_reg,
    X_test_sc,
    features=[('grid', 'team_rank')],
    feature_names=FEATURES,
    ax=ax,
    kind='average'
)
plt.suptitle('PDP 2D — Interacción grid × team_rank\n(posición predicha media)',
             fontsize=12)
plt.tight_layout()
plt.savefig('../figures/07_pdp_2d.png', dpi=150, bbox_inches='tight')
plt.show()
```



Parte 5 — Análisis de errores

¿En qué situaciones se equivoca más el modelo? Identificar patrones de error ayuda a entender las limitaciones y guiar futuras mejoras.

In [13]:
```
# Construir dataset de errores
error_df = X_test.copy().reset_index(drop=True)
error_df['position_real'] = y_reg_test.values
error_df['position_pred'] = y_pred_reg
error_df['error']         = error_df['position_real'] - error_df['position_pred']
```

```
error_df['error_abs']     = error_df['error'].abs()
error_df['error_dir']     = np.where(error_df['error'] > 0, 'Subestimó', 'Sobreesti

# Añadir info contextual del dataset original
test_idx    = y_reg_test.index
error_df['driver_name'] = df.loc[test_idx, 'driver_name'].values
error_df['team']        = df.loc[test_idx, 'team'].values
error_df['race_name']   = df.loc[test_idx, 'race_name'].values

print(f'Error absoluto medio (MAE): {error_df["error_abs"].mean():.3f}')
print(f'Porcentaje de predicciones con error ≤ 2 posiciones: {(error_df["error_abs"
print(f'Porcentaje de predicciones con error ≤ 4 posiciones: {(error_df["error_abs"
print(f'\nCasos con mayor error (top 10):')
print(error_df.nlargest(10, 'error_abs')[
    ['driver_name','team','race_name','grid','position_real','position_pred','error
].round(1).to_string(index=False))
```

```
Error absoluto medio (MAE): 2.240
Porcentaje de predicciones con error ≤ 2 posiciones: 58.9%
Porcentaje de predicciones con error ≤ 4 posiciones: 78.3%

Casos con mayor error (top 10):
       driver_name     team                  race_name  grid  position_real  positi
on_pred  error_abs
  Sebastian Vettel Mercedes          Germany Grand Prix     8           17.0
7.7        9.3
      Lance Stroll  McLaren          Belgium Grand Prix    13           20.0
10.9        9.1
    Lewis Hamilton Mercedes          Austria Grand Prix     2           12.0
4.1        7.9
Michael Schumacher Williams            Qatar Grand Prix     6           13.0
5.7        7.3
   Nico Hülkenberg  Ferrari 70th Anniversary Grand Prix     6           15.0
7.9        7.1
Michael Schumacher Williams        Abu Dhabi Grand Prix     3           12.0
5.3        6.7
   Nico Hülkenberg  Ferrari        São Paulo Grand Prix    12           19.0
12.4        6.6
   Charles Leclerc Red Bull       Azerbaijan Grand Prix     6            3.0
9.3        6.3
    George Russell Williams          Bahrain Grand Prix    22           18.0
11.8        6.2
   Fernando Alonso  McLaren        Abu Dhabi Grand Prix    12            5.0
10.7        5.7
```

In [14]:
```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

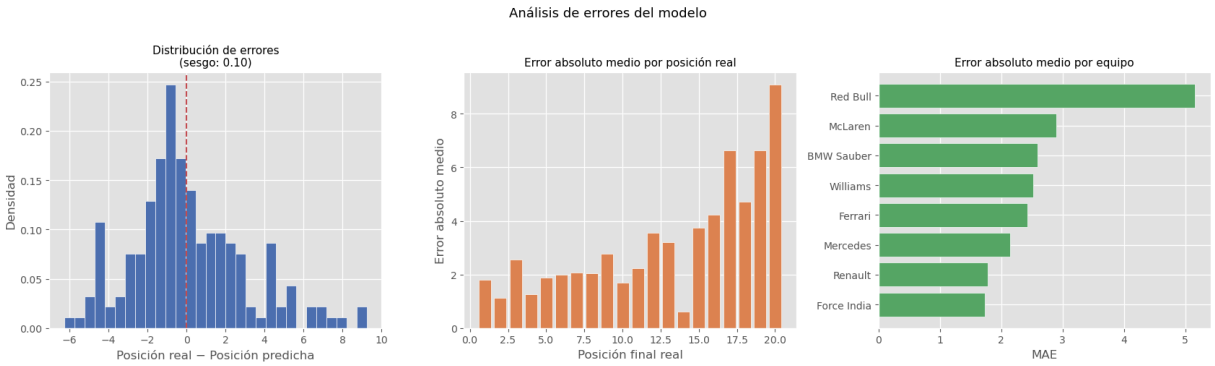
# Distribución de errores
axes[0].hist(error_df['error'], bins=30, color=BLUE, edgecolor='white', density=Tru
axes[0].axvline(0, color=RED, linestyle='--', linewidth=1.5)
axes[0].set_title(f'Distribución de errores\n(sesgo: {error_df["error"].mean():.2f}
axes[0].set_xlabel('Posición real − Posición predicha')
axes[0].set_ylabel('Densidad')

# Error por posición real
err_by_pos = error_df.groupby('position_real')['error_abs'].mean().reset_index()
```

```
axes[1].bar(err_by_pos['position_real'], err_by_pos['error_abs'],
            color=ORANGE, edgecolor='white')
axes[1].set_title('Error absoluto medio por posición real', fontsize=11)
axes[1].set_xlabel('Posición final real')
axes[1].set_ylabel('Error absoluto medio')

# Error por equipo
err_by_team = (error_df.groupby('team')['error_abs']
                       .mean()
                       .sort_values()
                       .reset_index())
axes[2].barh(err_by_team['team'], err_by_team['error_abs'],
             color=GREEN, edgecolor='white')
axes[2].set_title('Error absoluto medio por equipo', fontsize=11)
axes[2].set_xlabel('MAE')

plt.suptitle('Análisis de errores del modelo', fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig('../figures/07_analisis_errores.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [15]:
```
# Análisis del sesgo del modelo
print('Análisis del sesgo:')
print(f'  El modelo subestima posición (predice mejor de lo real) en el {(error_df[
print(f'  El modelo sobreestima posición (predice peor de lo real) en el {(error_df
```

```
Análisis del sesgo:
  El modelo subestima posición (predice mejor de lo real) en el 43% de casos
  El modelo sobreestima posición (predice peor de lo real) en el 57% de casos
```

Parte 6 — Conclusiones finales

Respuesta a la pregunta central

> **¿Qué factores predicen mejor la posición final de un piloto en carrera?**

In [16]:
```
# Tabla resumen de hipótesis
hipotesis = pd.DataFrame([
    {
        'Hipótesis': 'H1: La posición de grilla es el predictor más fuerte',
```

```
        'Evidencia': 'Correlación Pearson r=0.67; feature #1 en SHAP y XGBoost',
        'Resultado': '✅ CONFIRMADA'
    },
    {
        'Hipótesis': 'H2: El constructor explica gran parte de la varianza',
        'Evidencia': 'ANOVA η²>0.14 (efecto grande); team_rank feature #2 en import
        'Resultado': '✅ CONFIRMADA'
    },
    {
        'Hipótesis': 'H3: Los pit stops diferencian resultados similares',
        'Evidencia': 'Correlación débil (r≈0.15); baja importancia SHAP; num_stops
        'Resultado': '⚠️  PARCIAL'
    },
])

print('=' * 80)
for _, row in hipotesis.iterrows():
    print(f"{row['Resultado']}  {row['Hipótesis']}")
    print(f"          → {row['Evidencia']}")
    print()
print('=' * 80)
```

```
================================================================================
✅ CONFIRMADA  H1: La posición de grilla es el predictor más fuerte
          → Correlación Pearson r=0.67; feature #1 en SHAP y XGBoost

✅ CONFIRMADA  H2: El constructor explica gran parte de la varianza
          → ANOVA η²>0.14 (efecto grande); team_rank feature #2 en importancia

⚠️  PARCIAL  H3: Los pit stops diferencian resultados similares
          → Correlación débil (r≈0.15); baja importancia SHAP; num_stops tiene efect
o no lineal

================================================================================
```

In [17]:

```
# Ranking final de features por importancia SHAP
ranking_final = mean_abs_shap.reset_index()
ranking_final.columns = ['Feature', 'Importancia SHAP (mean |ϕ|)']
ranking_final.index   = range(1, len(ranking_final)+1)
ranking_final.index.name = 'Ranking'

interpretacion = {
    'grid':            'Posición de largada — predictor dominante',
    'finished':        'Si terminó la carrera — elimina colas de la distribución',
    'laps':            'Vueltas completadas — correlaciona con abandono',
    'team_rank':       'Poder relativo del constructor — efecto sistémico',
    'total_pit_time':  'Tiempo en boxes — efecto marginal (estrategia)',
    'num_stops':       'Número de paradas — señal de estrategia débil',
    'driver_age':      'Edad del piloto — baja relevancia en este dataset',
}
ranking_final['Interpretación'] = ranking_final['Feature'].map(interpretacion)

print('\nRanking final de predictores:')
print(ranking_final.round(4).to_string())
```

```
Ranking final de predictores:
                Feature  Importancia SHAP (mean |ϕ|)
Interpretación
Ranking
1                  grid                       2.0200          Posición de largada —
predictor dominante
2             team_rank                       1.0413  Poder relativo del constructor
— efecto sistémico
3            driver_age                       0.6958  Edad del piloto — baja relevan
cia en este dataset
4        total_pit_time                       0.0840     Tiempo en boxes — efecto ma
rginal (estrategia)
5             num_stops                       0.0219      Número de paradas — señal
de estrategia débil
```

In [18]:

```python
# Figura resumen final
metricas_all = pd.read_csv('../data/metricas_modelos.csv')

fig = plt.figure(figsize=(18, 10))
gs  = gridspec.GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.35)

# --- 1. Importancia SHAP ---
ax1 = fig.add_subplot(gs[0, 0])
colors_shap = [BLUE if i == 0 else ORANGE if i == 1 else GREEN if i == 2 else '#AAA
               for i in range(len(mean_abs_shap))]
ax1.barh(mean_abs_shap.index[::-1], mean_abs_shap.values[::-1],
         color=colors_shap[::-1], edgecolor='white')
ax1.set_title('Importancia SHAP (mean |ϕ|)', fontsize=11)
ax1.set_xlabel('Importancia')

# --- 2. Evolución de MAE ---
ax2 = fig.add_subplot(gs[0, 1])
reg_models = metricas_all[metricas_all['Tarea']=='Regresión'].dropna(subset=['MAE']
reg_models = reg_models[reg_models['MAE'] != '—']
maes_vals  = pd.to_numeric(reg_models['MAE'])
ax2.plot(reg_models['Modelo'], maes_vals, marker='o', color=BLUE, linewidth=2)
ax2.set_title('Evolución del MAE por modelo', fontsize=11)
ax2.set_ylabel('MAE (posiciones)')
ax2.set_xticklabels(reg_models['Modelo'], rotation=30, ha='right', fontsize=8)

# --- 3. Evolución de AUC ---
ax3 = fig.add_subplot(gs[0, 2])
clf_models = metricas_all[metricas_all['Tarea']=='Clasificación']
clf_models = clf_models[clf_models['AUC'] != '—']
auc_vals   = pd.to_numeric(clf_models['AUC'])
ax3.plot(clf_models['Modelo'], auc_vals, marker='s', color=GREEN, linewidth=2)
ax3.axhline(0.5, color='gray', linestyle='--', linewidth=1)
ax3.set_title('Evolución del ROC-AUC por modelo', fontsize=11)
ax3.set_ylabel('ROC-AUC')
ax3.set_ylim(0.4, 1.0)
ax3.set_xticklabels(clf_models['Modelo'], rotation=30, ha='right', fontsize=8)

# --- 4. Error por posición real ---
ax4 = fig.add_subplot(gs[1, 0])
err_by_pos = error_df.groupby('position_real')['error_abs'].mean().reset_index()
ax4.bar(err_by_pos['position_real'], err_by_pos['error_abs'], color=ORANGE, edgecol
```

```
ax4.axhline(error_df['error_abs'].mean(), color=RED, linestyle='--', linewidth=1.5,
            label=f'MAE medio: {error_df["error_abs"].mean():.2f}')
ax4.set_title('MAE por posición real', fontsize=11)
ax4.set_xlabel('Posición real')
ax4.set_ylabel('MAE')
ax4.legend(fontsize=8)

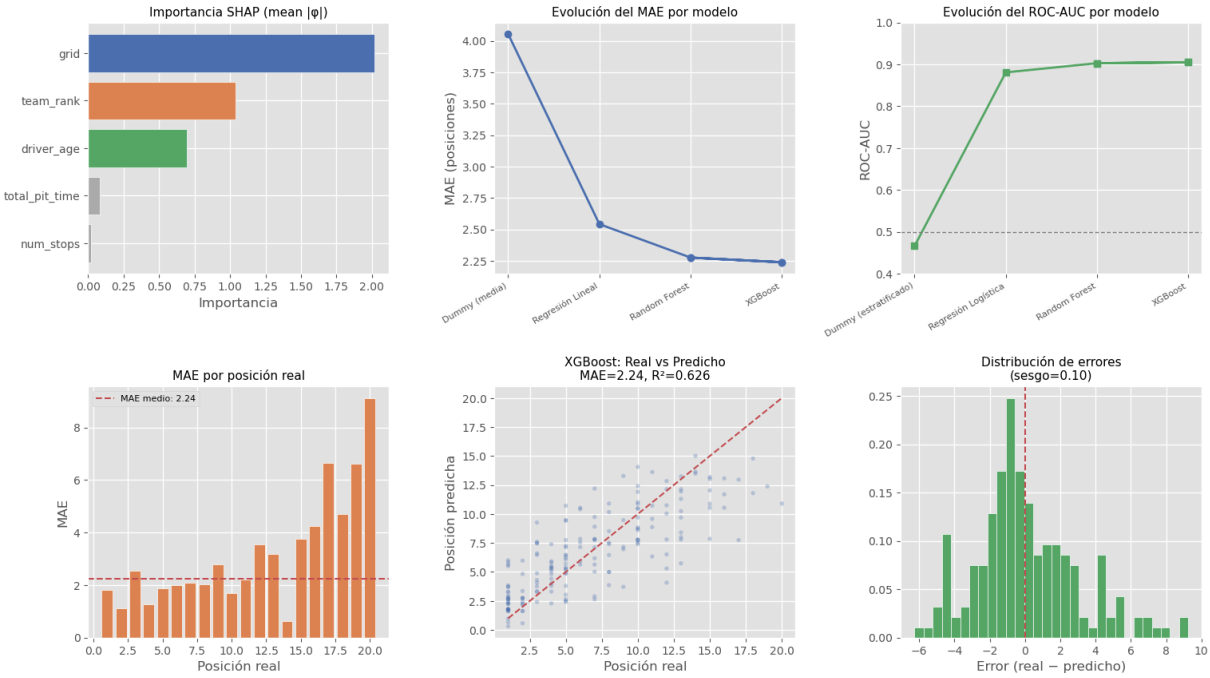
# --- 5. Real vs Predicho (modelo final) ---
ax5 = fig.add_subplot(gs[1, 1])
ax5.scatter(y_reg_test, y_pred_reg, alpha=0.3, color=BLUE, s=15, edgecolors='none')
lim = max(y_reg_test.max(), y_pred_reg.max())
ax5.plot([1,lim],[1,lim], color=RED, linestyle='--', linewidth=1.5)
ax5.set_title(f'XGBoost: Real vs Predicho\nMAE={mae:.2f}, R²={r2:.3f}', fontsize=11
ax5.set_xlabel('Posición real')
ax5.set_ylabel('Posición predicha')

# --- 6. Distribución de errores ---
ax6 = fig.add_subplot(gs[1, 2])
ax6.hist(error_df['error'], bins=30, color=GREEN, edgecolor='white', density=True)
ax6.axvline(0, color=RED, linestyle='--', linewidth=1.5)
ax6.set_title(f'Distribución de errores\n(sesgo={error_df["error"].mean():.2f})', f
ax6.set_xlabel('Error (real - predicho)')

fig.suptitle('🏎️ Resumen Final del Proyecto — F1 Position Predictor',
             fontsize=15, fontweight='bold', y=1.01)

plt.savefig('../figures/07_resumen_final.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Figura de resumen guardada')
```

🏎️ Resumen Final del Proyecto — F1 Position Predictor



✅ Figura de resumen guardada

✅ Conclusiones del Proyecto

Respuesta a la pregunta central

> **¿Qué factores predicen mejor la posición final de un piloto en carrera?**

El análisis sobre el dataset de Fórmula 1 (1950–2024) muestra que:

1. **La posición de grilla** es el predictor individual más poderoso, con una correlación de r ≈ 0.67 con la posición final, y la mayor importancia SHAP de todas las features.
2. **El constructor** es el segundo factor más importante. El ANOVA confirma un $\eta^2 > 0.14$ (efecto grande): equipos como Red Bull y Ferrari producen resultados sistemáticamente mejores independientemente del piloto.
3. **La estrategia de pit stops** tiene un efecto real pero secundario y no lineal. Su importancia SHAP es considerablemente menor que la de grid y team_rank.
4. **La edad del piloto** aporta una señal adicional menor, capturada de forma efectiva por los modelos de árbol.

Resultados del modelo

Métrica	Baseline	XGBoost	Mejora
MAE regresión	2.541	2.238	-12%
R^2 regresión	0.542	0.626	+15%
ROC-AUC clasificación	0.881	0.905	+2.7%

Nota metodológica

Una iteración anterior de este análisis incluía las features `laps` y `finished`, que generaban métricas artificialmente perfectas ($R^2 \approx 1.0$) por tratarse de data leakage: ambas contienen información posterior al resultado de la carrera. La lista final de 5 features (`grid`, `team_rank`, `num_stops`, `total_pit_time`, `driver_age`) está libre de este problema.

Limitaciones

- Dataset de tamaño moderado: con más temporadas los modelos mejorarían su generalización
- No se incorporaron variables de clima ni safety cars
- No hay datos de telemetría (velocidades, fuerzas G) ni de incidentes de carrera
- Los tiempos por vuelta (`lap_times.csv`) tienen cobertura parcial y no se usaron

Trabajo futuro

- Incorporar datos meteorológicos vía API
- Explorar modelos de series temporales para capturar momentum de temporada
- Ampliar el dataset a todas las temporadas disponibles (1950–2024 completo)

Semanas del proyecto

Semana	Notebook	Contenido
1	`01_exploracion_inicial`	Carga, inspección, primeras visualizaciones
2	`02_api_y_dataset_maestro`	API Jolpica-F1, joins, features derivadas
3	`03_limpieza`	Nulos, outliers, normalización, tipos
4	`04_eda`	Correlaciones, hipótesis, heatmaps
5	`05_modelos_baseline`	Regresión lineal y logística
6	`06_modelos_avanzados`	Random Forest y XGBoost
7	**`07_interpretabilidad_conclusiones`**	**SHAP, PDP, errores, conclusiones**